LONDON METROPOLITAN UNIVERSITY

ITAHARI INTERNATIONAL COLLEGE

Module Code & Module Title
CS5053NT - Cloud Computing and IOT

Assessment Type
50% coursework

Semester
2025 Autumn, 3rd semester

Assignment Submission Date: January 2, 2025
Submitted To: Anish Pandit
Word Count: 6151

I confirm that I understand my coursework needs to be submitted online via Google Classroom under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Acknowledgement

We would like to thank all the individuals who have supported us and our team throughout this Smart Greenhouse project. Also, colleges have provided us with a great learning environment and facilities.

First, I would like to express my genuine gratitude for our supervisor **Mr. Ishan kafle** and **Mr. Sonam Rai** and our module leader **Mr. Anish Pandit** for their guidance and constructive feedback. Their suggestions helped us in leading our project to success. Their support helped us to understand the concept of IOT and all the sensors or things that we have used in implementing our projects.

We would also like to thank our team leader Ms Akisha Bhujel for guiding and managing our team throughout the coursework, and all the members for co-operating and giving their full efforts to make our Smart Greenhouse Project successful. With our unity and hard work, we were able to complete this coursework. With equal contribution from each member our Smart Greenhouse was fully possible.

Abstract

As the need to have effective and sustainable agricultural practices, the incorporation of Internet of Things (IOT) technology has been explored as a good option that could enhance the growth of plants and greenhouses. The conventional greenhouse systems have been very dependent on manual supervision to control the environmental parameters of temperature, humidity, soil moisture, and light intensity. The time-consuming nature of this manual process, its susceptibility to human error, and the fact that there will be inconsistent growth of the plants because of the delay in responding to varying environmental conditions makes it a time-consuming method. To overcome these obstacles, the proposed project is a Smart Greenhouse System using the IoT to provide a mechanism of automated monitoring and control of the key parameters of the environment needed to allow the plants to grow healthy.

The proposed system will use a set of sensors such as temperature and humidity, soil moisture and light sensors as the system will always receive real time data on the environment in the greenhouse. The main processing unit is the ESP32 micro-controller, on which sensor data are introduced and smart decisions are made within the set threshold values. Anytime the environment falls out of the desirable ranges, the system will automatically activate the respective actuators like cooling fans, water pumps and artificial lighting in order to create a balance within the environment. The extent of reduction of the necessity of the human presence at all times is high in such automation as it will guarantee the controlled and sustainable environment of growing plants.

The system will have cloud connectivity in terms of MQTT based communication to provide convenience and remotely monitor the system. This sensor data and system performance must be transferred to a cloud system and displayed in a convenient web or mobile interface. This will allow users to monitor the situation in greenhouses in real time, manually switch the device on or off where it is necessary and receive a notice or a warning in case some of the crucial environmental parameters have become

excessively high or excessively low. Such and other properties enable timely action and avoid the loss of plants in case of the drastic alterations in the environment.

Altogether, the Smart Greenhouse System based on IOT is expected to enhance the health of plants, optimize the use of resources, and eliminate extra labor aimed at offering the modern agricultural sphere with the reliable, automated, and scalable system. The project shows how the IOT technology can be successfully used at small scale greenhouse systems and provide a low-cost and viable solution to gardeners, farmers and agricultural enthusiasts. Moreover, this system also provides the basis of the further development of smart agriculture, enhancing the efficiency of water management, providing constant control, and decision-making based on data.

Table of contents

Table of tables 10
1. Introduction 11
1.1. Current scenario 1
1.2. Problem Statement and Project as a solution 1
1.2. Aims and Objectives 2
Aims: 2
Objectives: 2
2.1. System Overview 3
2.2 Design Diagram 3
2.2.1 Block Diagrams 4
2.2.2 Flowchart 6
2.2.4 Schematics Diagram 9
2.3. Requirement Analysis 10
2.3.1. Hardware Components 10
2.3.2. Software components 16
3. Development 16
3.1 planning and Design 16
3.2 Resource Allocations 16
3.3 System Development 16
3.3.1 For Temperature Monitoring: 16
3.3.2. For Fan Control 17
3.3.3 For LCD Display 18
3.3.4 For Soil Moisture Monitoring 18
3.3.5. For water pump 18
3.4 Log Book 19
4. Result and Finding 24
4.1 Testing no.1 - LCD displays text unsuccessfully 24
4.2 Test no. 2 - DHT22 temperature used to check temperature unsuccessful 25
4.3 Test no.3 - DHT22 sensor used to check temperature unsuccessful 26
4.4 Test no.4 - LCD display text 27
4.7 Test no.7 - soil sensor dry test 32

4.8 Test no.8 - Change in temperature 34
4.9 Test no.9 - Change in Humidity 36
4.10 Test no.10 - Relay click test 38
5. Future work 39
6. Conclusion 40
7. References 41
8. Appendix 42
8.1. Appendix A: Source code 42
8.1.1. main.py 42
8.1.2. Lcd_api.py 46
8.1.3. Machine_i2c_lcd.py 54
8.1.4. Lcd_test.py 57
8.2. Appendix B: Testing & Supporting Materials 59
2.2. system overview 59
2.2.1 Block Diagrams 59
2.2.2 Flowchart 60
2.2.3. Circuits diagram 61
2.2.4 Schematics Diagram 62
2.3.2. Software components 63
3.1 planning and Design 65
3.2 Resource Allocations 66
3.3 System Development 67
3.3.1 For Temperature Monitoring: 67
3.3.2. For Fan Control 68
3.3.3 For LCD Display 71
3.3.4 For Soil Moisture Monitoring 72
3.3.5. For water pump 73
4. Result and Finding 76
4.11. Test no.11 - LCD values updates 76
4.13 Test no.13 – Successful pump on test 79
4.14 Test no.14 – Unsuccessful pump off test 81
4.15 Test no.15 – Successful pump off test 82
4.16 Test no.16 –Unsuccessful fan on 83

4.17 Test no.17–Successful fan on test 85
4.18 Test no.18 – Successful fan off test 86
4.20. Test no.20 - automatic greenhouse 89
4. Future work 93
5. Conclusion 94

Table of Figure

Figure 1 : Block Diagram 4
Figure 2 System Architecture 5
Figure 3 : Flow chart 7
Figure 4 Circuits Diagram 8
Figure 5 : Schematics Diagram 9
Figure 6 : ESP 32 10
Figure 7 : DHTT 22 Sensor 10
Figure 8 : Moisture Sensor 11
Figure 9 : Relay Module 11
Figure 10 : Jumper Wires 12
Figure 11 : Half Breadboard 12
Figure 12 : Water Motor 13
Figure 13 : Moter Fan 13
Figure 14 : Battery 14
Figure 15 : LCD display 14
Figure 16 : Potentiometer 15
Figure 17 : USB cable 15
Figure 18 : Resistor 15
Figure 19 : Log 1 19
Figure 20 : Log 2 20
Figure 21 : Log 3 21
Figure 22 : Log 4 22
Figure 23 : Log 5 23
Figure 24 : LCD display unsucessful 24
Figure 25 : DHT22 temperature unsucessful 25
Figure 26 : DHT22 humidity unsuccessful 26
Figure 27 : LCD display test 27
Figure 28 : DHT22 sensor to check temperature 28
Figure 29 : LCD displays temperature 28
Figure 30 : DHT22 sensor thonny 29

Figure 31 : DHT22 sensor to check temperature *30*
Figure 32 : LCD displays temperature *30*
Figure 33 : DHT22 sensor Humidity thonny code. *31*
Figure 34 : soil dry test *32*
Figure 35 : soil value in thonney *33*
Figure 36 : change in temperature *34*
Figure 37 : lighter used to raise temperature *35*
Figure 38 : temperature raised *35*
Figure 39 : change in humidity *36*
Figure 40 : blow in DHT22 to change humidity *37*
Figure 41 : humidity decreased *37*
Figure 42 : when relay is activated *38*
Figure 43 : when relay is deactivated *38*
Figure 44 : Thonny IDE logo *63*
Figure 45 : actual setup of temperature sensor *68*
Figure 46 fan and relay module *69*
Figure 47 actual setup of fan development *70*
Figure 48 ESP32 and LCD connection *71*
Figure 49 actual setup of LCD display *72*
Figure 50 : actual setup of soil sensor *73*
Figure 51 : actual setup of water pump *74*
Figure 52 : Final smart green house *75*
Figure 53 : initial testing displayed on LCD *76*
Figure 54 : final testing displayed on LCD *77*
Figure 55 : unsuccessful pump on test *78*
Figure 56 : Successful pump on *80*
Figure 57 : Unsuccessful pump off test *81*
Figure 58 : sucessful moter test *83*
Figure 59 : Unsuccessful fan on *84*
Figure 60 : Successful fan test *86*
Figure 61 : Successful fan off *87*

Figure 62 : Node Red MQTT 88
Figure 63 : Humidity Graph 89
Figure 64 : temperature graph 89
Figure 65 : soil sensor in greenhouse 90
Figure 66 : fan turn on when temperature reach 24°c 91
Figure 67 : oil sensor in greenhouse water motor turn on when soil is dry 91
Figure 68 : water motor turn on when soil is dry 92
Figure 69 : smart greenhouse 92

Table of tables

Table 1 : LCD displays text unsuccessfully testing *24*
Table 2 : DHT22 sensor testing *25*
Table 3 : DHT22 sensor Unsuccessful *26*
Table 4 : LCD display text Testing *27*
Table 5 : DHT22 sensor successful *28*
Table 6 : DHT22 sensor humidity Testing *30*
Table 7 : soil sensor dry test *32*
Table 8 : Change in temperature *34*
Table 9 : Change in humidity *36*
Table 10 : Relay click test *38*
Table 11 : LCD values updates Testing *76*
Table 12 : Unsuccessful pump on test *77*
Table 13 : Successful pump on test *79*
Table 14 : Unsuccessful pump off test *81*
Table 15 : Successful pump off testing *82*
Table 16 : Unsuccessful fan on testing *83*
Table 17 : successful fan on testing *85*
Table 18 : Successful fan off testing *86*
Table 19 : complet automatic greenhouse overview *89*

1. Introduction

In current years, the implementation of smart automatic devices has been becoming increasingly significant in enhancing our day-to-day activities, incorporating agricultural and backyard gardening. The traditional way requires a lot of heavy manuals, and a lot of effort which can result in time consumption and inconsistency. With a growing interest in IOT based efficient solutions, the internet offers a modern way to monitor and handle environmental conditions. These projects focus on building a smart greenhouse system which relies on IOT technology to generate an easy-to-use environment for plants growth.

1.1. Current scenario

At present time, most people review plant conditions by ground moisture, temperature and moisture in the air by hand. These inspections are usually predicted, which can lead to excess watering, poor ventilation and inconsistent development. Since the climate becomes less reliable and more gardeners in limited space, there is a requirement for a convenient, affordable, self-operating system which facilitates plants optimal growth with minimal effort.

1.2. Problem Statement and Project as a solution

The primary challenge in greenhouses is absence of an easy and effective medium to maintain plants even in an optimal environment through minimal observation. Manual supervision could lead to inconsistent watering, slow reactions to alteration in temperature or moisture, and difficulty in keeping the plant surrounding steady. Our Smart Greenhouse project addresses this via help of sensors, self-operating devices, and IOT technology. It tracks heat level, air moisture, and soil moisture, with self-operating air circulation and water supply through a fan and pump. This approach provides a steady and high performing plant growing conditions, minimizing day-to-day chores, and assisting plants to remain healthy.

1.2. Aims and Objectives

1.2.1. Aims:

The aim of this project is to make a smart greenhouse which helps in solving the problem of human supervision by continuously checking and monitoring the environmental condition of the greenhouse and controlling all the devices which are required for the better growth of the plants.

1.2.2. Objectives:

i. To check the environmental condition using sensors to monitor the temperature, humidity, moisture of the soil and lights.
ii. The smart green house should be able to turn on and off devices like fans and lights so the system can control devices without the help of humans.
iii. Sending data on a web dashboard or mobile allowing the user to see the condition of the greenhouse.
iv. To give alerts and warnings if the value becomes too high or low.

2. Background

2.1. System Overview

By the end of this project, we will be able to have a fully functional Smart Greenhouse System built upon IoT that will have the ability to automatically determine and regulate the essential environmental factors to help the growth of plants.
appendix

2.2 Design Diagram

A design diagram is a graphical illustration of a system, which displays the key elements of the system, their connections, and the data flow or control between them. A design diagram is significant since it assists in visualizing the way a system is organized in a clear and structured manner and it assists in the visualization of the interaction between the components of a system. It simplifies complicated concepts, enhances collaboration between the members of the team, and assists in organizing the system prior to its adoption. The design diagrams minimize mistakes, save time and the documentation is more professional and clearer to evaluate.

2.2.1 Block Diagrams

The block diagram is a depiction of a monitoring and control system incorporating IOT that will gauge the environmental conditions and operate the devices automatically. The system consists of a DHT22 sensor to measure and monitor temperatures and humidity and a soil moisture sensor to measure water content in soil. appendix

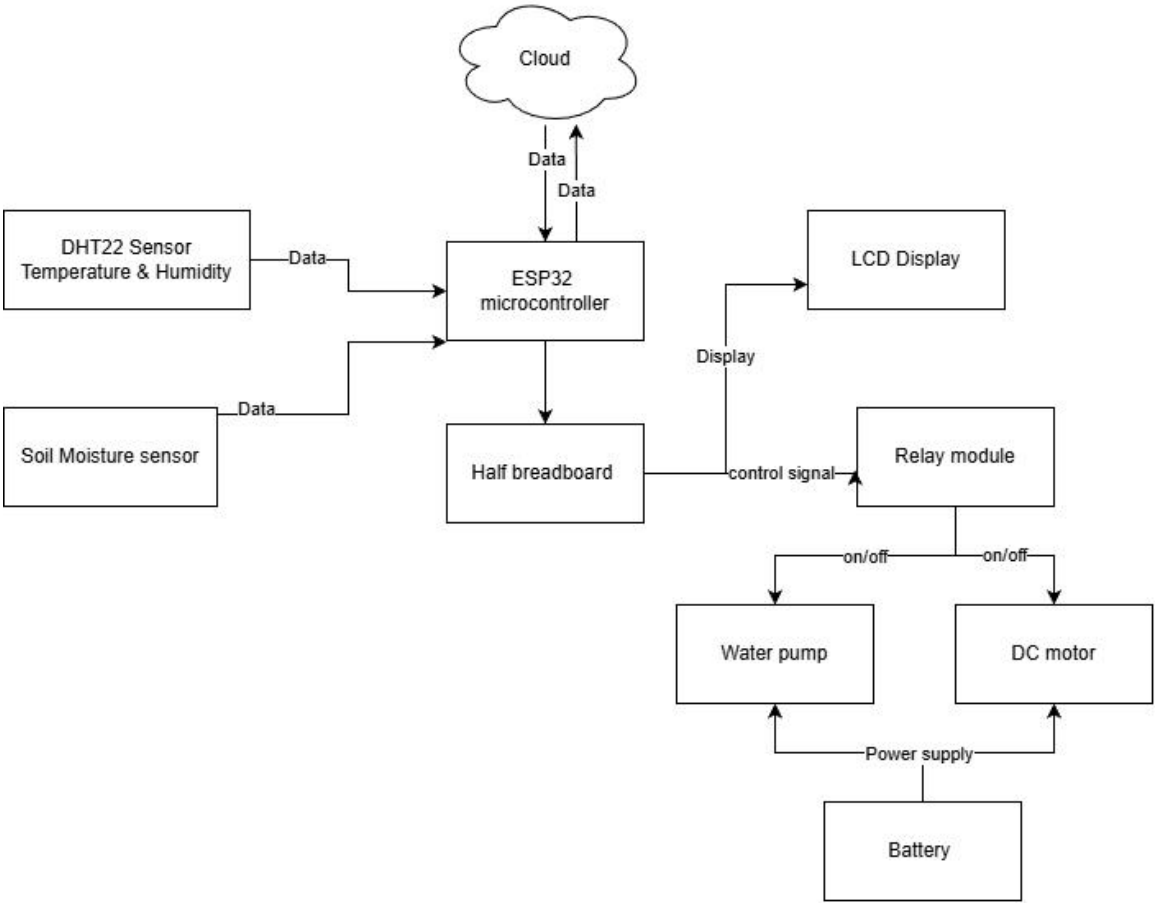


Figure 1: Block Diagram

2.2. System Architecture

The physical structure of a system that presents hardware components in the system and also their connection to work together is the hardware architecture of a system, which is significant since it helps in understanding of the system connection, correct implementation, minimization of errors, and clear description of the system design in documentation.

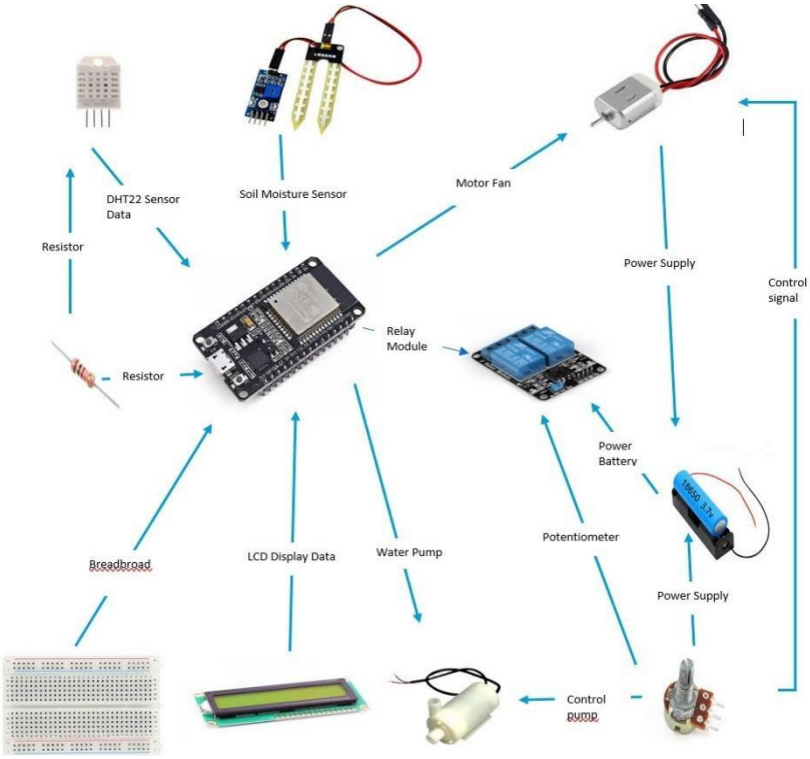


Figure 2 System Architecture

2.2.2 Flowchart

Flowcharts are important in modeling complex systems in a simple and structured way. Flowcharts simplify the process of determining logic of the system, tracing the flow of data, and detecting possible errors in planning, implementation, and debugging phases by using standardized symbols like arrows, rectangles, and decision diamonds. Appendix

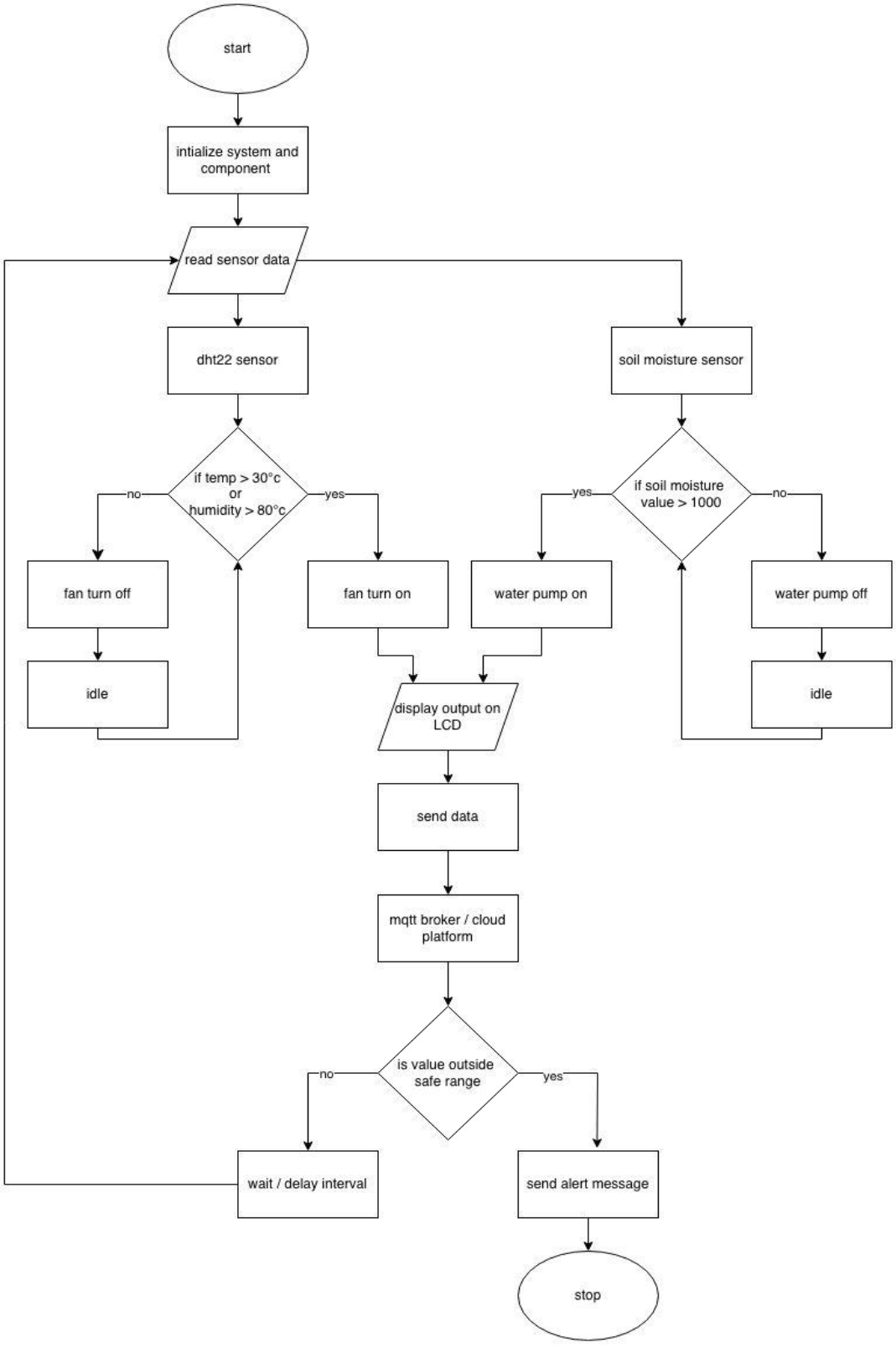


Figure 3: Flow chart

2.2.3. Circuits diagram

The presented circuit is an IOT-based automated system of plant watering (smart watering) constructed on an ESP32 micro-controller. The ESP32 is the primary controller, which receives the input of other sensors (soil moisture sensor (to determine the level of wetness or dryness of the soil) and the DHT sensor (to determine the temperature and humidity) at the same time. Appendix

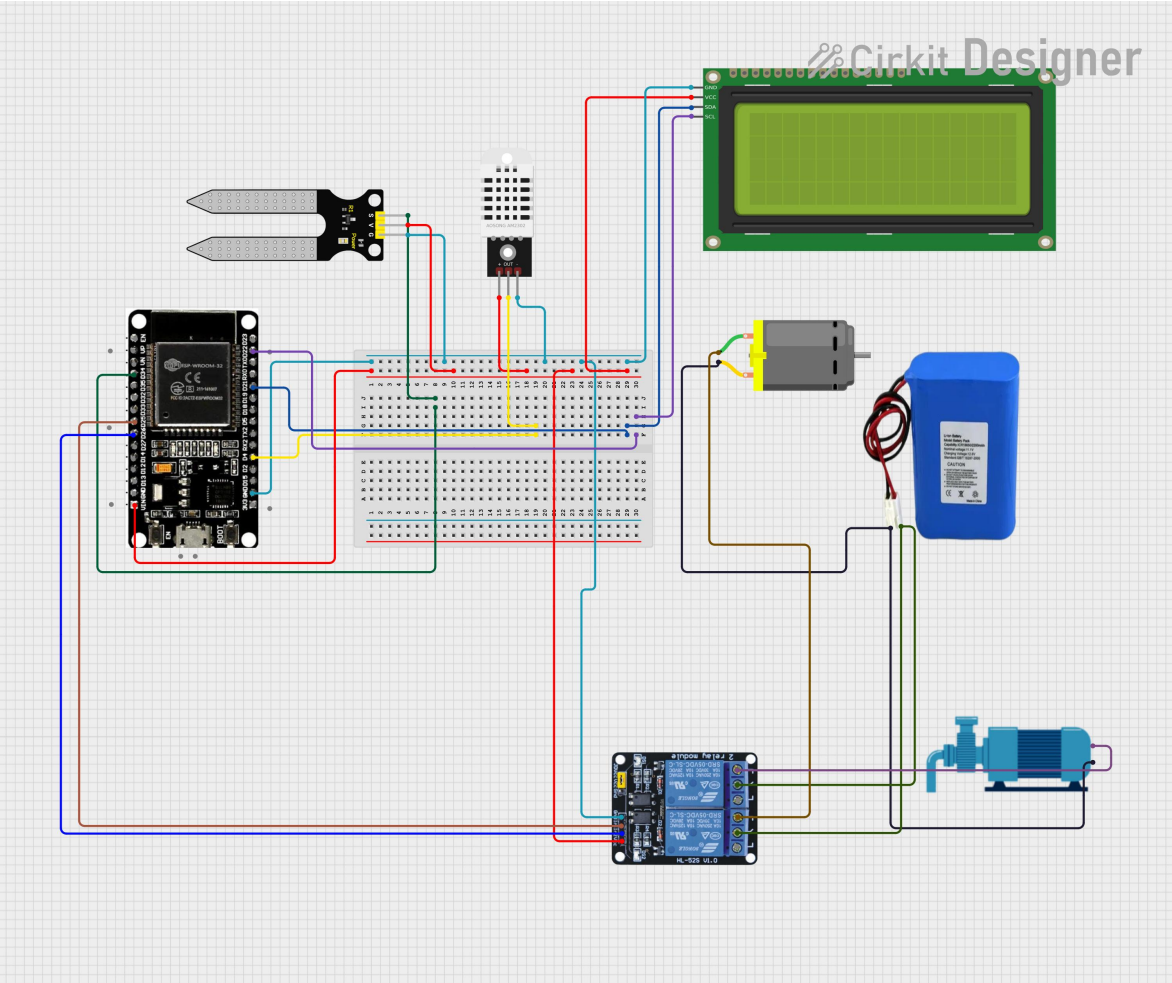


Figure 4 Circuits Diagram

2.2.4 Schematics Diagram

A schematic diagram refers to a generalized, graphic representation of an electrical circuit as an electrical graph that shows the relationships between the various components of an electrical circuit. Appendix

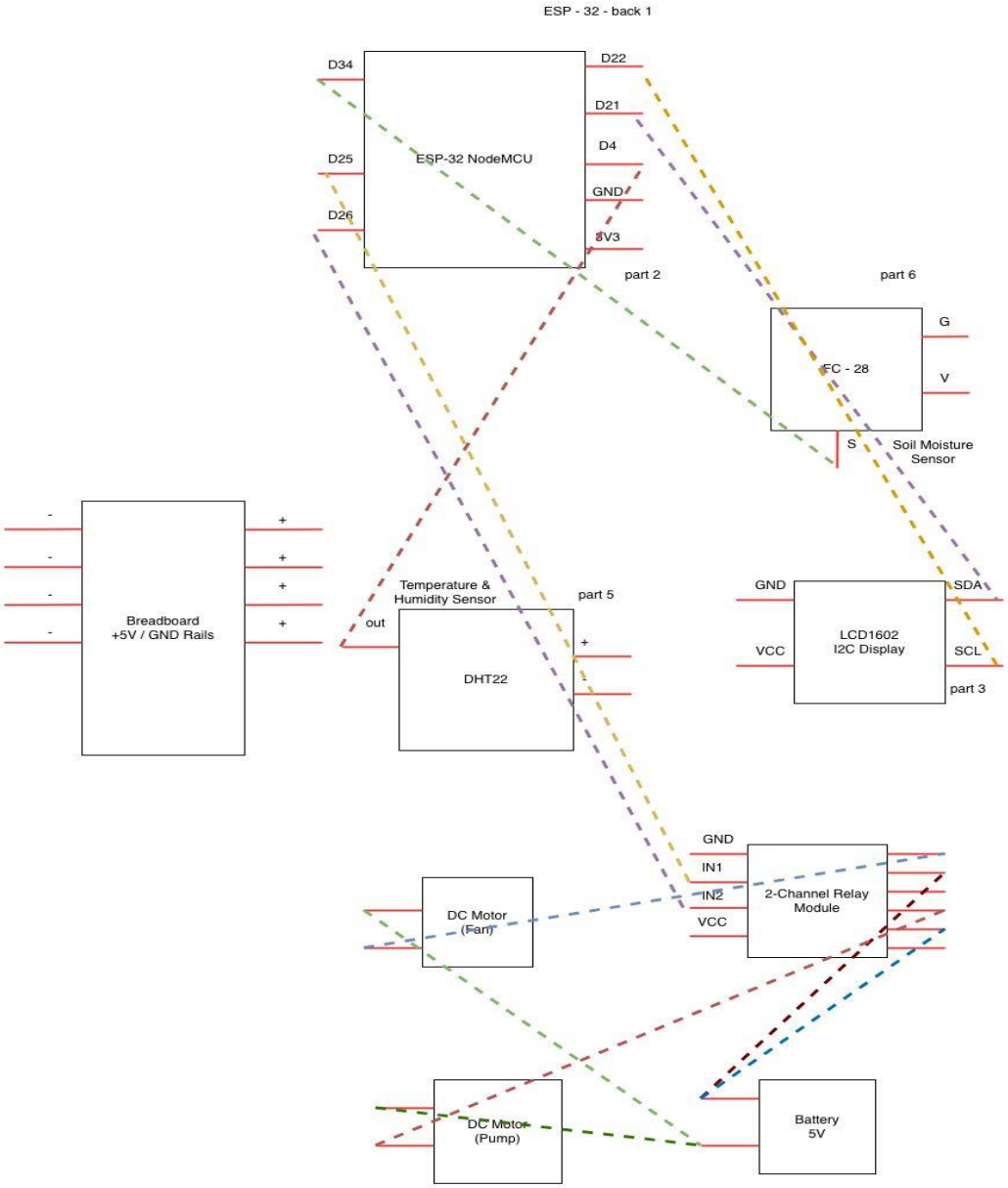


Figure 5: Schematics Diagram

2.3. Requirement Analysis

2.3.1. Hardware Components

i. **ESP32:** The ESP32 micro-controller was used as a central processing unit of the system which collected sensor data, performed control logic and allowed wireless communication. The Smart Greenhouse system is controlled by a computer called the ESP32. It reads data sent out by sensors and then processes it in order to make decisions. It also has Wi-Fi that supports monitoring by use of the Internet of Things.

Figure 6: ESP 32

ii. **DHT22 Sensor:** The system uses the DHT22 Sensor to control things. The motors and pumps are operated automatically by the system which uses temperature and humidity sensor to control the system. These are values applied in maintenance of proper environmental conditions during plant growth. This would allow it to switch on the motor fan and the water pump when required. The DHT22 has much higher accuracy and stability as compared to simple sensors, so it can be used in monitoring green houses.

Figure 7: DHTT 22 Sensor

iii. **Soil Moisture Sensor:** It is a device that measures the amount of water in the soil. It assists in determining whether the soil is dry or is moist enough. According to the sensor readings, the ESP32 is used to switch the water pump on and off to water plants only when there is a need. The soil moisture thing prevents the plant from getting much water or not enough water.

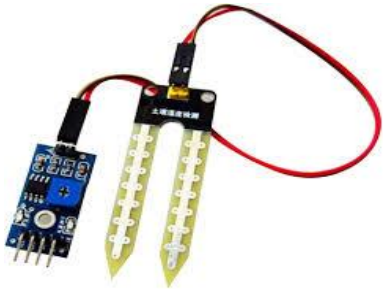


Figure 8: Moisture Sensor

iv. **Relay Module:** It enables a low-power supply circuit to switch on or regulate a high-power supply circuit without integrating it with the same circuit. The relay module enhances the safety of the system and avoids microcontroller damage. It is an essential part of the smart greenhouse system because its guarantees effective management of power and stable operation of external devices.

Figure 9: Relay Module

v. **Jumper Wires (Male to Female):** Jumper wires are electric connections that are used to make the connections between the circuit elements. These jumper wires are quite handy as they allow passing the signals and power between the ESP32, the sensors and the actuators. It is not necessary to solder the ESP32 to the sensors and the actuators which makes things easier. They can be used in testing

and implementing simple and flexible connections of circuits. Jumper wires give flexibility and they also save a lot of time when working on a project.

Figure 10: Jumper Wires

vi. **Half Breadboard:** A breadboard is a base circuit to build a reusable circuit board on a temporary basis. It is easy to place components on it and very easy to modify them during testing. It enables electronic parts to be inserted and interconnected easily without the need to have permanent connections. The half breadboard is quite handy as the board can have parts taken and added to it and vice versa.

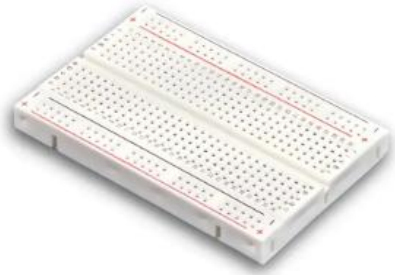


Figure 11: Half Breadboard

vii. **Water Motor (Pump):** The water motor is an electronic which is employed to pump water through a storage to the plants. It is operated by a relay module which automatically switches it off and on. This way the water motor helps to keep the soil moist and saves people from having to water the plants all the time. The water motor also helps to prevent wasting water. Irrigation is made automatic by use of water pump.

Figure 12: Water Motor

viii. **Motor Fan:** The motor fan is used to manage the temperature and air circulation inside the greenhouse. It aids in the maintenance of the internal environment by lowering the amount of heat and enhancing ventilation which is fundamental in the proper growth of plants. The motor fan in the smart greenhouse system will be under the control of the ESP32 microcontroller through a relay module. The motor fan is turned on or off automatically depending on what the temperature and humidity sensors, in the DHT22 sensor readings.

Figure 13 : Moter Fan

ix. **Battery (3.7V):** The smart greenhouse is powered by a 3.7v battery. It is the main power source, for the smart greenhouse system. It is very portable and always easy to operate particularly when there is power outage. The smart greenhouse system can be used in outdoor locations with no continuous power supply with this battery.

Figure 14: Battery

x. **LCD display:** LCD is used to refer to Liquid Crystal Display. LCD display provides the user with real-time visual feedback. To enable the user interaction and local monitoring, an LCD display was installed to show real-time data such as temperature, humidity, the level of soil moisture and status of the system. The LCD enhances usability because people are able to keep track of greenhouse conditions in their area without the use of the internet.

Figure 15: LCD display

xi. **Potentiometer:** A potentiometer is an electronic element that is a variable voltage divider and you can regulate the output voltage. Potentiometer was added to enable people to adjust the control parameters like fan speed or threshold values inside the potentiometer, which is flexible during testing and operation.

Figure 16: Potentiometer

xii. **USB port:** A USB cable is inserted into the computer to connect the esp32 to the computer. This connection is mandatory in the uploading of programs, configuration of the system, and in serial communication. It is used to monitor and debug the system in real-time. This is quite useful, to determine whether or not the esp32 and the sensors are functioning properly.

Figure 17: USB cable

xiii. **Resistor:** The pull-up resistors that are applied to the sensor, e.g., the DHT22, and the current-limiting elements are the primary categories of resistors in this project. It puts a current limitation on a circuit to ensure no sensitive component is destroyed. They assist in maintaining constant transmission of signals and stable working of the circuit and make the system as a whole safer and more accurate.

Figure 18: Resistor

2.3.2. Software components

Appendix

3. Development

3.1 planning and Design

The Greenhouse Monitoring and Control System design and planning was done in an organized way in 16 days. This stage was aimed at establishing the purpose of the system, the needs, the choice of appropriate elements, and the creation of a working circuit to be utilized. Appendix

3.2 Resource Allocations

The IOT Smart Greenhouse project was successfully developed only after the selective distribution of different hardware and software resources. The components were chosen according to their availability, compatibility with ESP32 and operational ability in a greenhouse setup. appendix

3.3 System Development

3.3.1 For Temperature Monitoring:

Phase 1: Connecting Temperature Sensor into the System

- Firstly, Esp32 is connected to Breadboard than the VIN pin of Esp32 is connected to positive and DND is connected to negative. Also, DHT22 positive pin is connected to breadboard positive and negative is connected to breadboard of negative.
- To begin with, the temperature sensor (which was DHT22) was set up onto the breadboard, The out pin of DHTT22 is connected to pin D4 of Esp32.

Appendix

3.3.2. For Fan Control

Phase 1: Fan and Relay Module relation.

- Firstly, to provide the possibility of safe switching, the fan was connected to the relay module.
- The relay module was connected to the breadboard containing ESP32 micro-controller.

Appendix

3.3.3 For LCD Display

Phase 1: Connecting LCD and the Esp32 Micro-controller.

- firstly, the LCD sensor is attached on the breadboard.

Appendix

3.3.4 For Soil Moisture Monitoring

Phase 1: Connecting the soil moisture senor to esp32 Module

- ESP32 is connected with breadboard and jumper wires are used to connect relay module, LCD.

Appendix

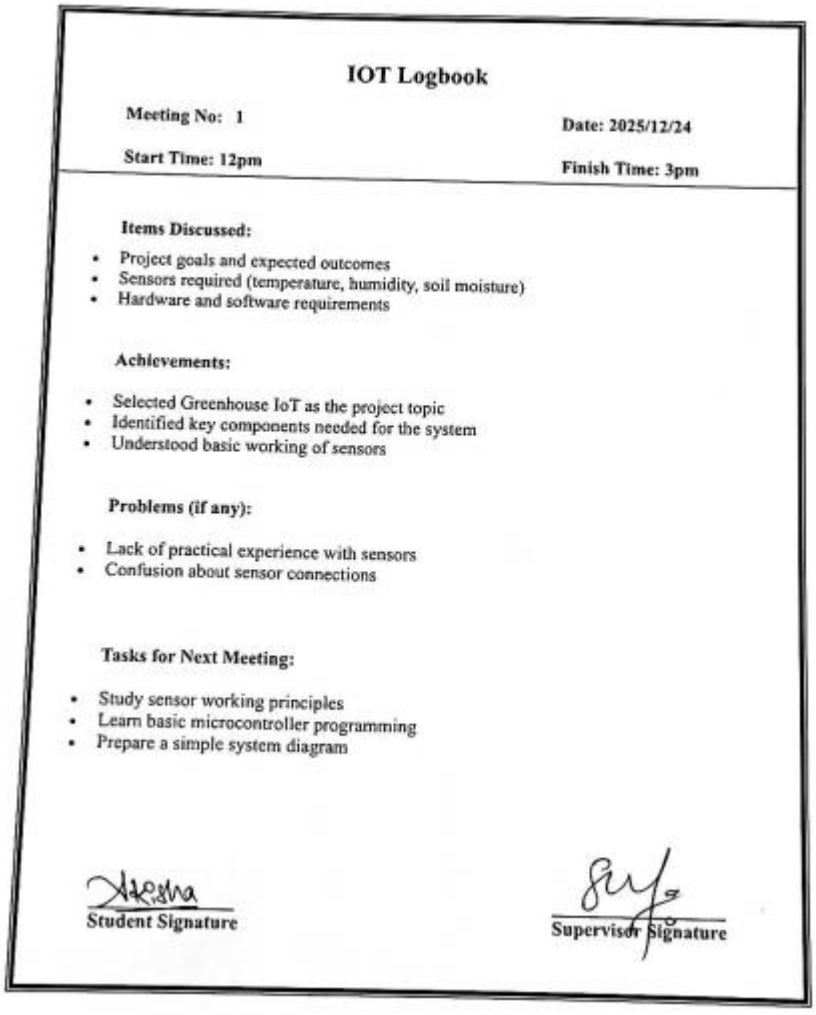
3.3.5. For water pump

Phase 1: Water pump and Relay Module relation.

- The water pump wire (+) positive is connected to the relay module COM; it helps in controlling the water pump with switching the power on and off according to the logic and condition of soil.

Appendix

3.4 Log Book



IOT Logbook

Meeting No: 1 **Date: 2025/12/24**

Start Time: 12pm **Finish Time: 3pm**

Items Discussed:

- Project goals and expected outcomes
- Sensors required (temperature, humidity, soil moisture)
- Hardware and software requirements

Achievements:

- Selected Greenhouse IoT as the project topic
- Identified key components needed for the system
- Understood basic working of sensors

Problems (if any):

- Lack of practical experience with sensors
- Confusion about sensor connections

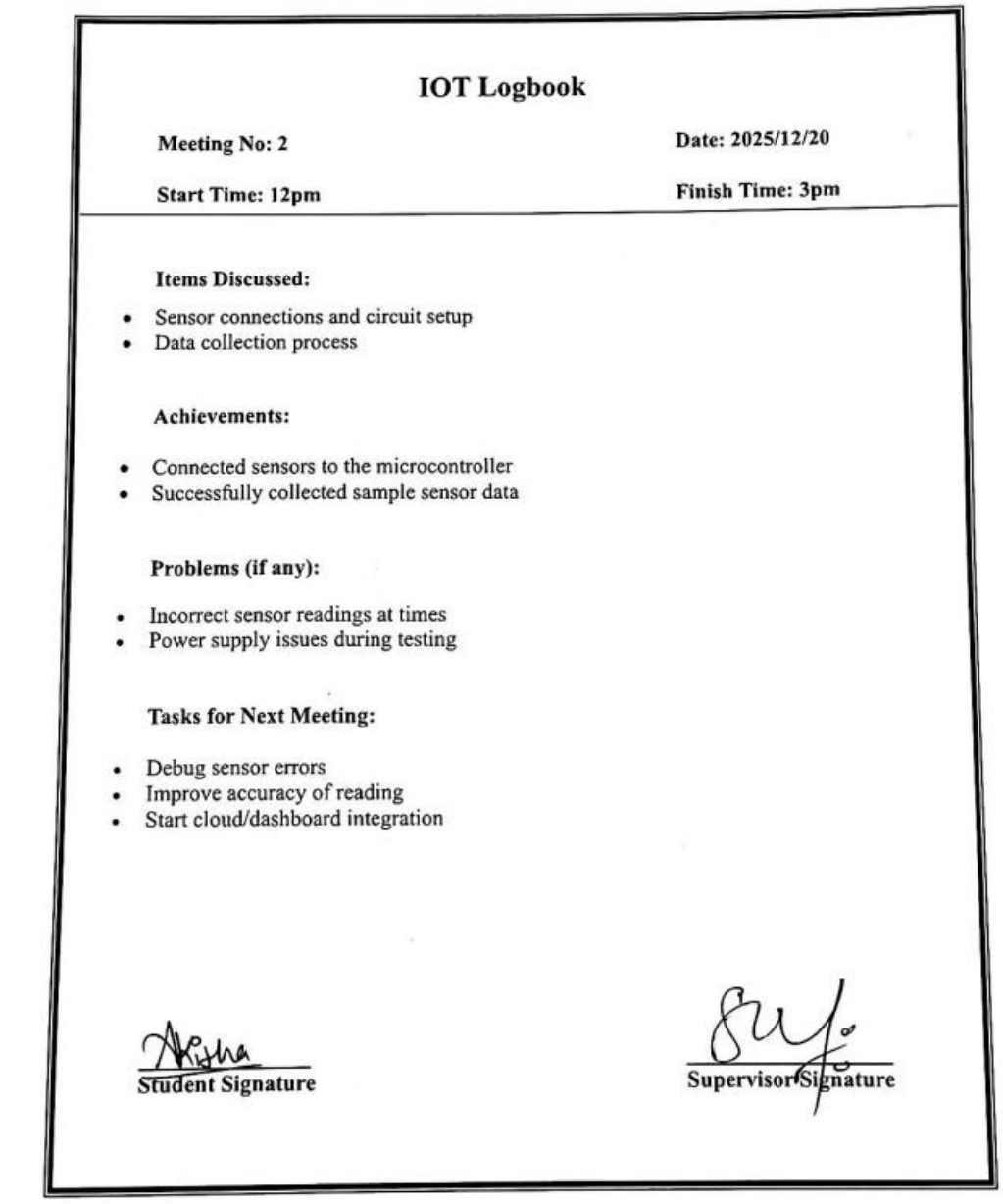
Tasks for Next Meeting:

- Study sensor working principles
- Learn basic microcontroller programming
- Prepare a simple system diagram

Student Signature

Supervisor Signature

Figure 19: Log 1



IOT Logbook

Meeting No: 2 **Date: 2025/12/20**

Start Time: 12pm **Finish Time: 3pm**

Items Discussed:

- Sensor connections and circuit setup
- Data collection process

Achievements:

- Connected sensors to the microcontroller
- Successfully collected sample sensor data

Problems (if any):

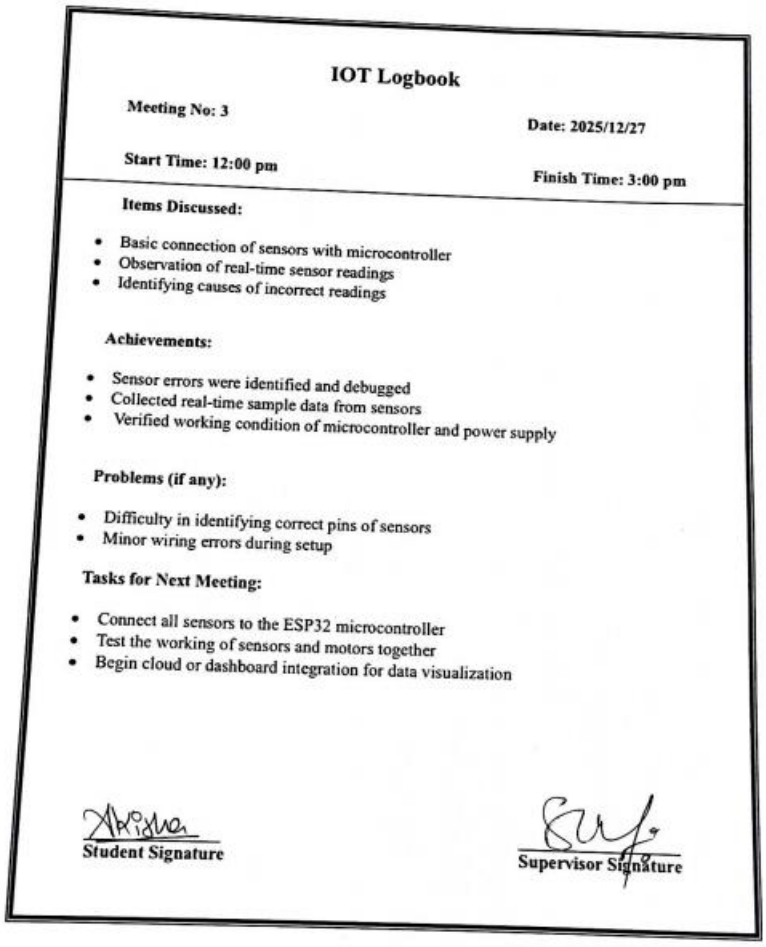
- Incorrect sensor readings at times
- Power supply issues during testing

Tasks for Next Meeting:

- Debug sensor errors
- Improve accuracy of reading
- Start cloud/dashboard integration

Student Signature **Supervisor Signature**

Figure 20: Log 2



IOT Logbook

Meeting No: 3 **Date: 2025/12/27**

Start Time: 12:00 pm **Finish Time: 3:00 pm**

Items Discussed:

- Basic connection of sensors with microcontroller
- Observation of real-time sensor readings
- Identifying causes of incorrect readings

Achievements:

- Sensor errors were identified and debugged
- Collected real-time sample data from sensors
- Verified working condition of microcontroller and power supply

Problems (if any):

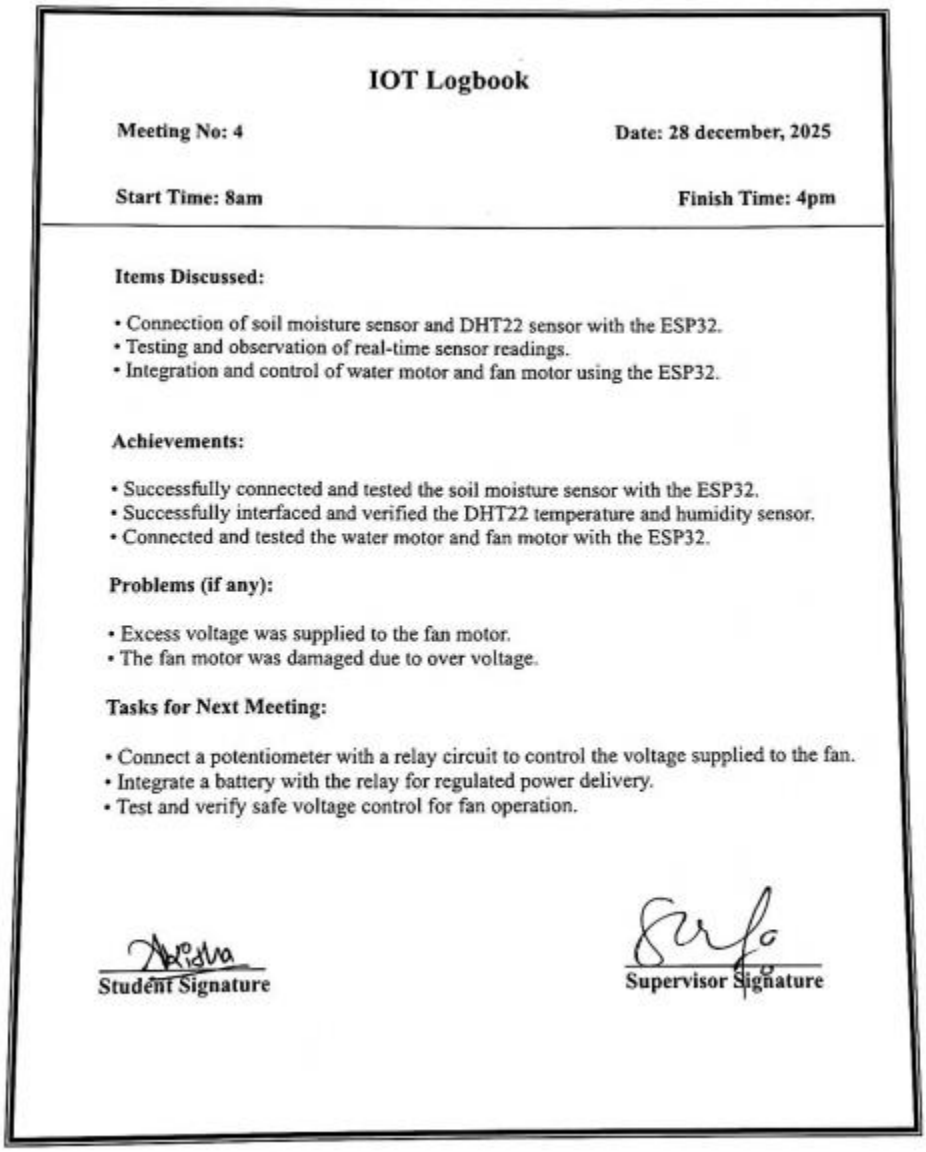
- Difficulty in identifying correct pins of sensors
- Minor wiring errors during setup

Tasks for Next Meeting:

- Connect all sensors to the ESP32 microcontroller
- Test the working of sensors and motors together
- Begin cloud or dashboard integration for data visualization

Student Signature **Supervisor Signature**

Figure 21: Log 3



IOT Logbook

Meeting No: 4 **Date: 28 december, 2025**

Start Time: 8am **Finish Time: 4pm**

Items Discussed:

- Connection of soil moisture sensor and DHT22 sensor with the ESP32.
- Testing and observation of real-time sensor readings.
- Integration and control of water motor and fan motor using the ESP32.

Achievements:

- Successfully connected and tested the soil moisture sensor with the ESP32.
- Successfully interfaced and verified the DHT22 temperature and humidity sensor.
- Connected and tested the water motor and fan motor with the ESP32.

Problems (if any):

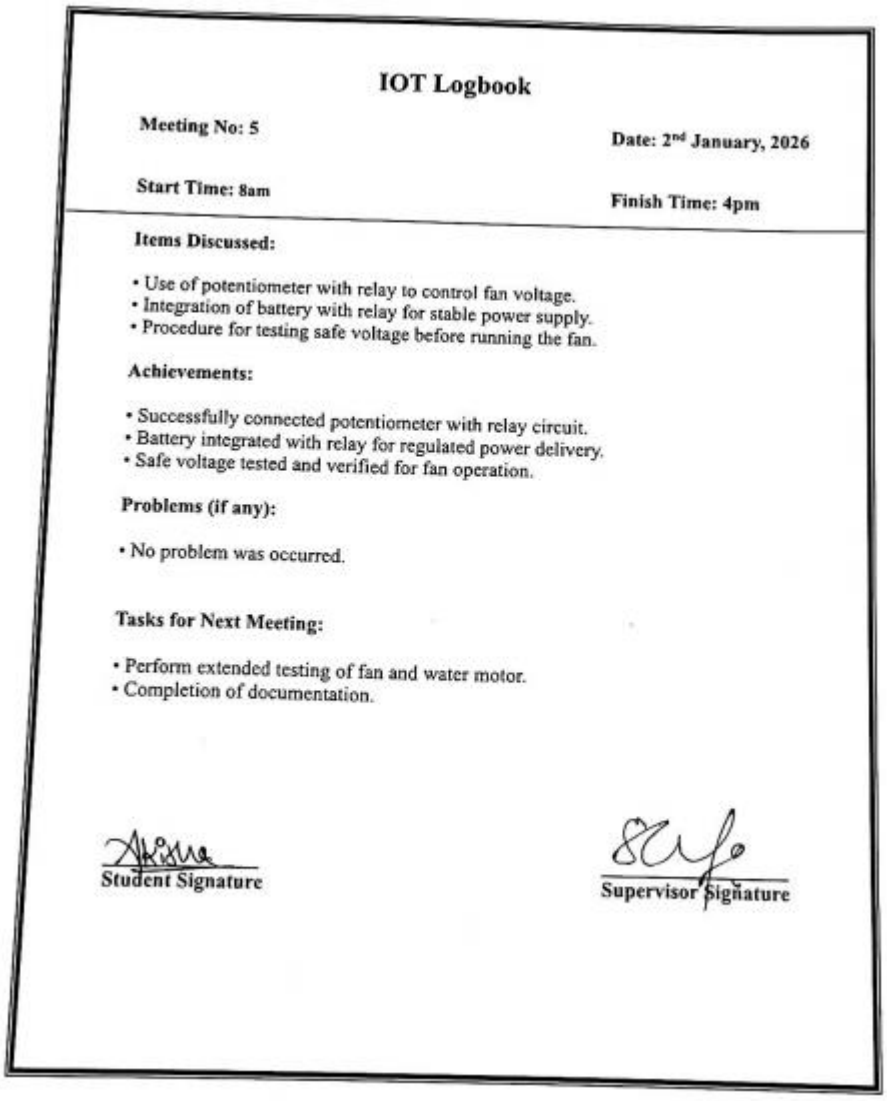
- Excess voltage was supplied to the fan motor.
- The fan motor was damaged due to over voltage.

Tasks for Next Meeting:

- Connect a potentiometer with a relay circuit to control the voltage supplied to the fan.
- Integrate a battery with the relay for regulated power delivery.
- Test and verify safe voltage control for fan operation.

Student Signature **Supervisor Signature**

Figure 22: Log 4



IOT Logbook

Meeting No: 5 **Date: 2nd January, 2026**

Start Time: 8am **Finish Time: 4pm**

Items Discussed:

- Use of potentiometer with relay to control fan voltage.
- Integration of battery with relay for stable power supply.
- Procedure for testing safe voltage before running the fan.

Achievements:

- Successfully connected potentiometer with relay circuit.
- Battery integrated with relay for regulated power delivery.
- Safe voltage tested and verified for fan operation.

Problems (if any):

- No problem was occurred.

Tasks for Next Meeting:

- Perform extended testing of fan and water motor.
- Completion of documentation.

Student Signature **Supervisor Signature**

Figure 23: Log 5

4. Result and Finding

4.1 Testing no.1 - LCD displays text unsuccessfully

Table 1: LCD displays text unsuccessfully testing

Objectives	Use an LCD to display values.
Action	Upload code on system and display in LCD.
Expected results	LCD displays correct data.
Actual results	The LCD did not display any data.
conclusion	The test was unsuccessful.

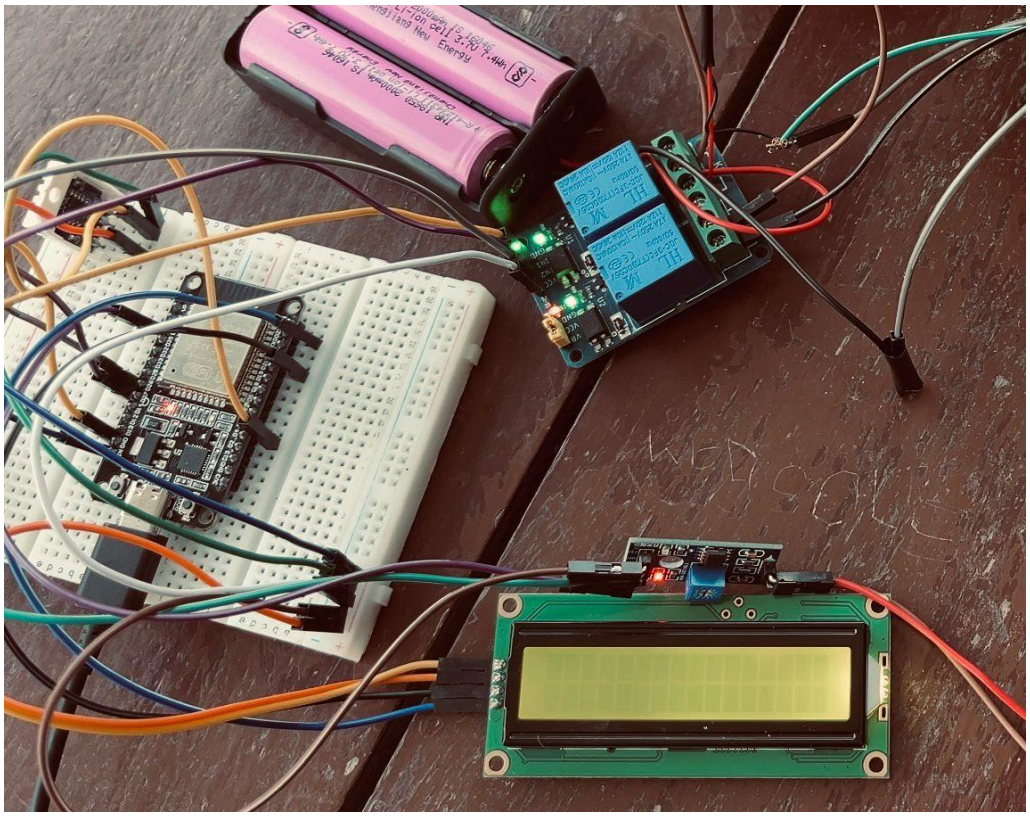


Figure 24: LCD display unsucessful

4.2 Test no. 2 - DHT22 temperature used to check temperature unsuccessful

Table 2: DHT22 sensor testing

Objectives	Use DHT22 sensor to check temperature.
Action	Connect sensor and run code.
Expected results	LCD displays correct data.
Actual results	LCD did not display any data
conclusion	The test was unsuccessful.

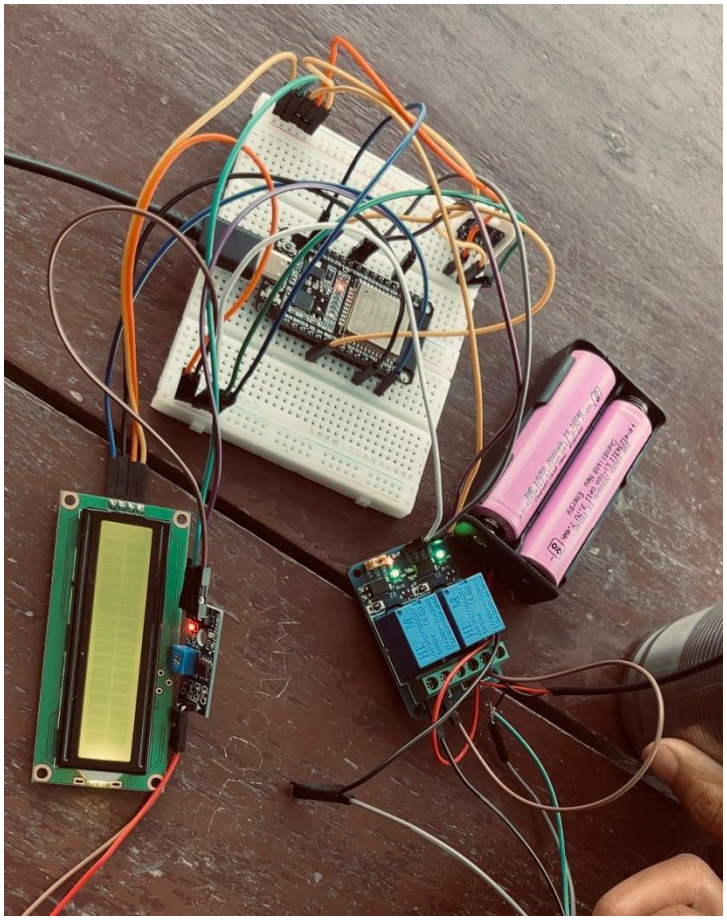


Figure 25: DHT22 temperature unsucessful

4.3 Test no.3 - DHT22 sensor used to check temperature unsuccessful

Table 3: DHT22 sensor Unsuccessful

Objectives	Use DHT22 sensor to check temperature.
Action	Connect sensor and run code.
Expected results	LCD displays correct data.
Actual results	LCD did not display any data
conclusion	The test was successful.

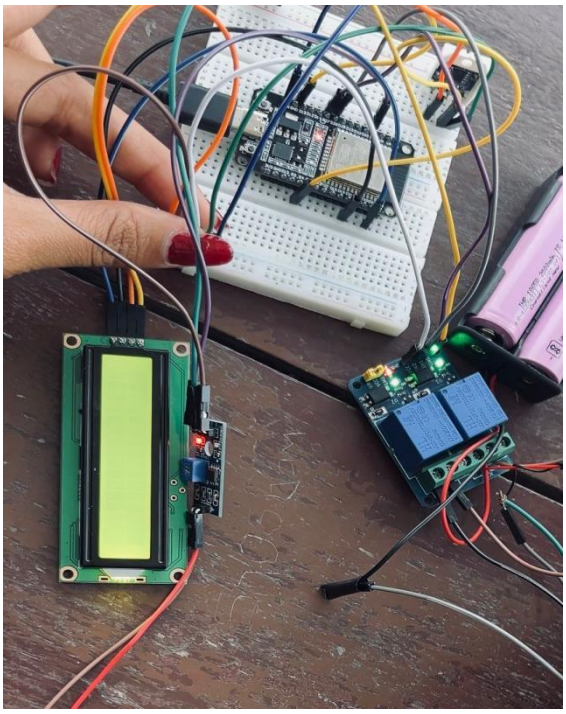


Figure 26: DHT22 humidity unsuccessful

4.4 Test no.4 - LCD display text

Table 4: LCD display text Testing

Objectives	Use an LCD to display values.
Action	Upload code on system and display in LCD
Expected results	LCD displays correct data
Actual results	LCD displayed correct value
conclusion	The test was successful.

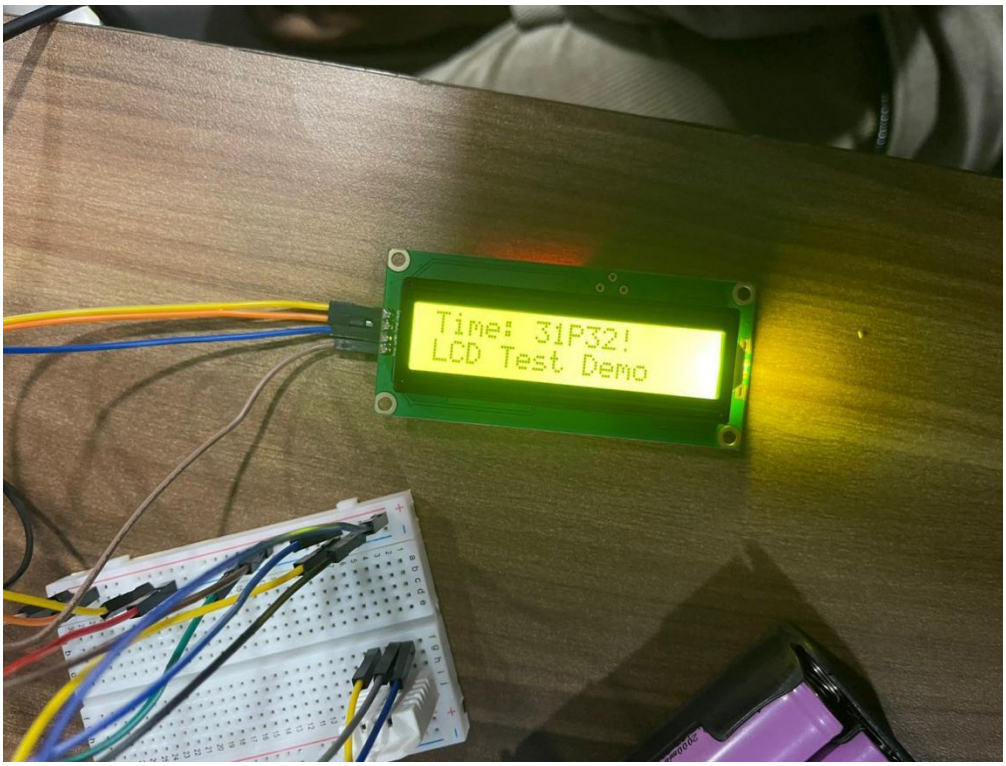


Figure 27: LCD display test

4.5 Test no.5 - DHT22 sensor used to check temperature

Table 5: DHT22 sensor successful

Objectives	Use DHT22 sensor to check temperature.
Action	Connect sensor and run code.
Expected results	The sensor gives out room temperature.
Actual results	The sensor gave room temperature.
conclusion	The test was successful.

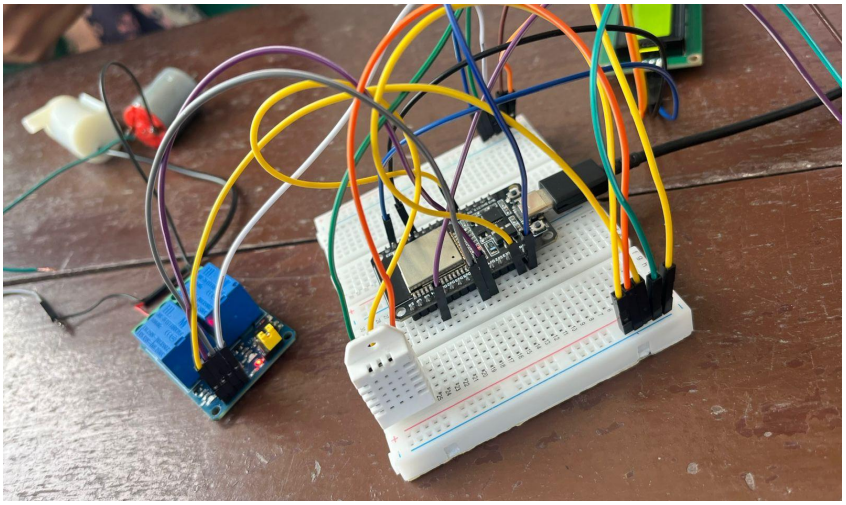


Figure 28: DHT22 sensor to check temperature

Figure 29: LCD displays temperature

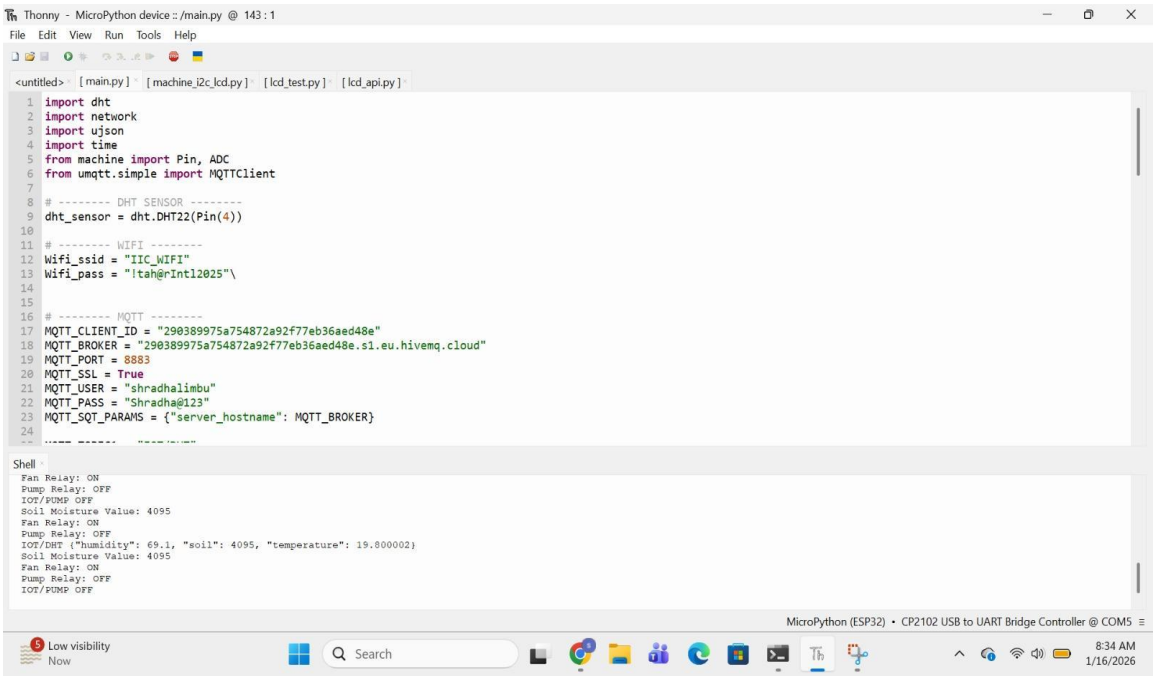


Figure 30: DHT22 sensor thonny

4.6 Test no.6 - DHT22 sensor used to check humidity

Table 6: DHT22 sensor humidity Testing

Objectives	Use a DHT22 sensor to check humidity.
Action	Connect sensor and run code.
Expected results	The sensor gives out humidity.
Actual results	The sensor gave humidity.
conclusion	The test was successful.

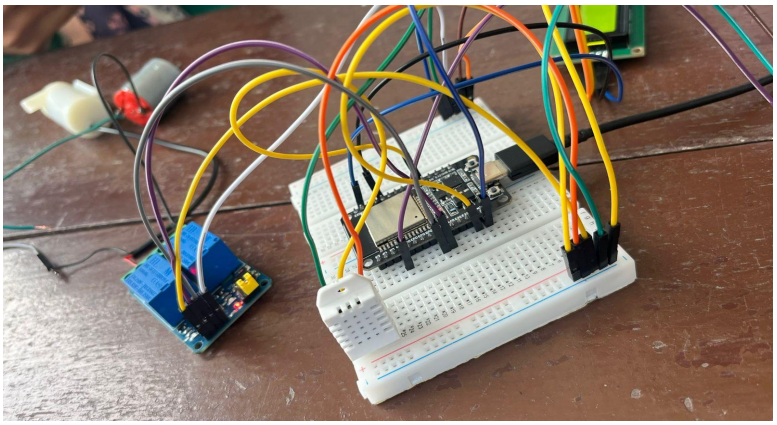


Figure 31: DHT22 sensor to check temperature

i

Figure 32: LCD displays temperature

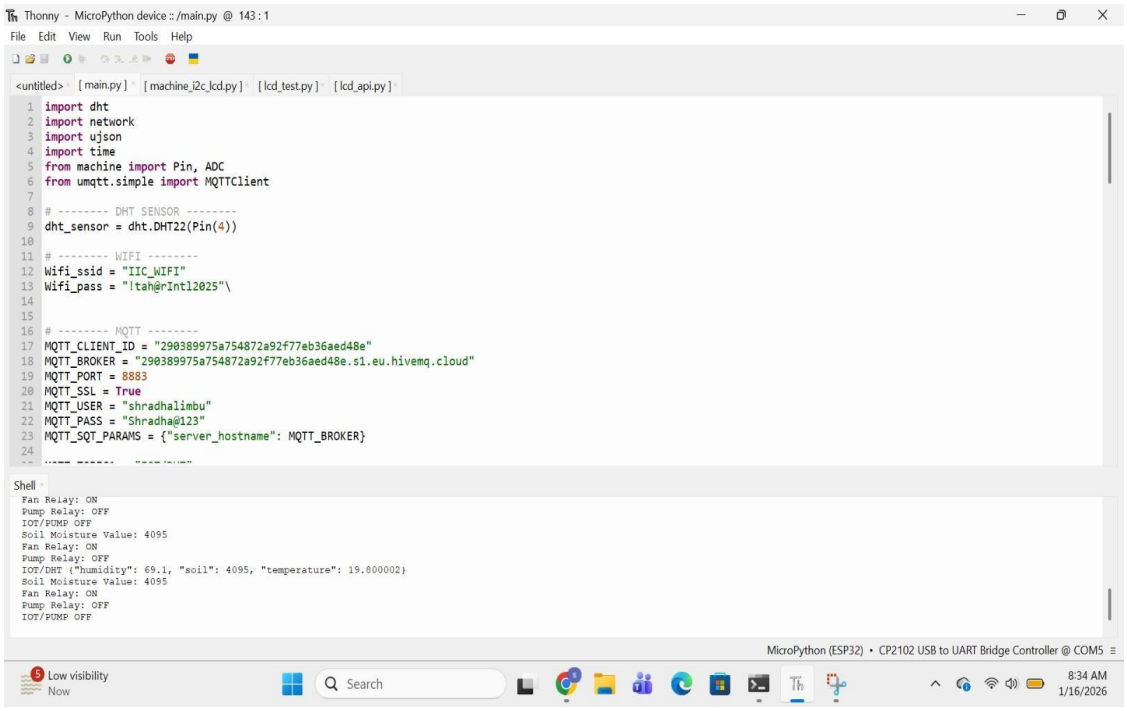


Figure 33: DHT22 sensor Humidity thonny code.

4.7 Test no.7 - soil sensor dry test

Table 7: soil sensor dry test

Objectives	Use a soil sensor to test on dry soil.
Action	Read observation of soil.
Expected results	The sensor shows dry value.
Actual results	The sensor showed dry value.
conclusion	The test was successful.

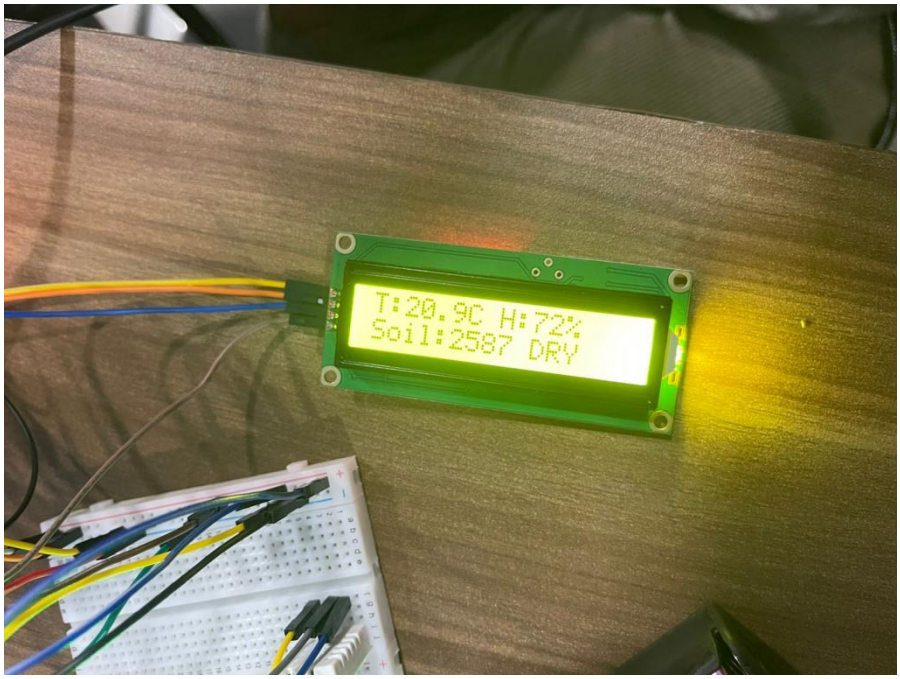


Figure 34: soil dry test

Figure 35: soil value in thonney

4.8 Test no.8 - Change in temperature

Table 8: Change in temperature

Objectives	Use a lighter to raise the temperature of the sensor.
Action	Lighter taken near to raise the temperature.
Expected results	The sensor shows new value.
Actual results	The sensor showed new value.
conclusion	The test was successful.

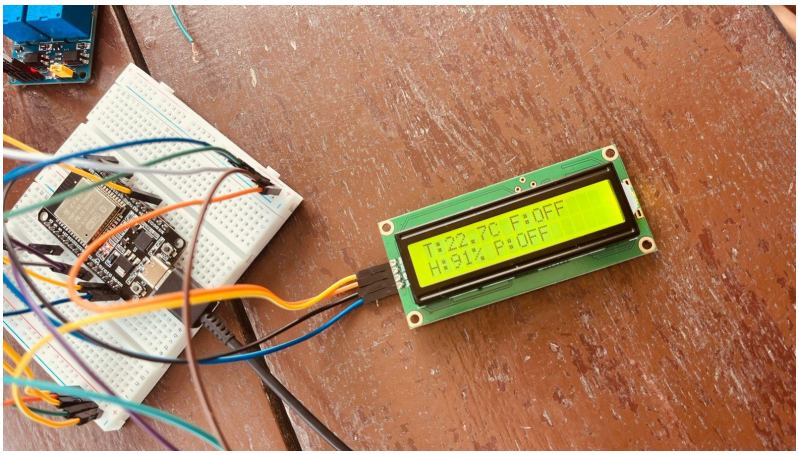


Figure 36: change in temperature

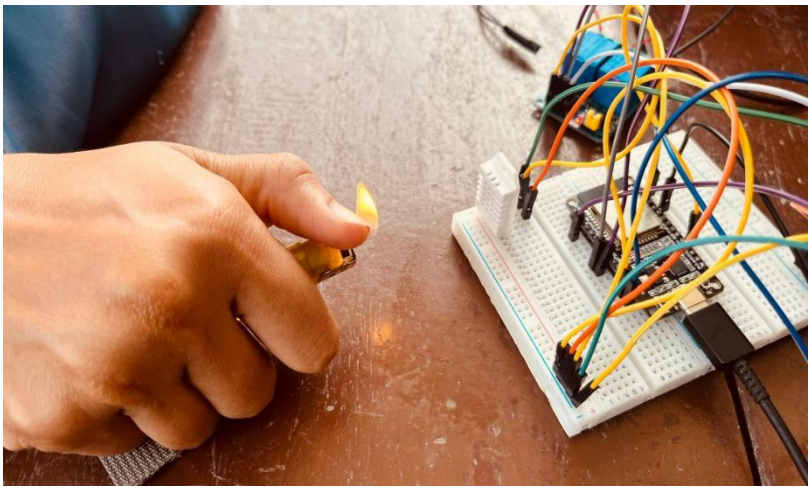


Figure 37: lighter used to raise temperature

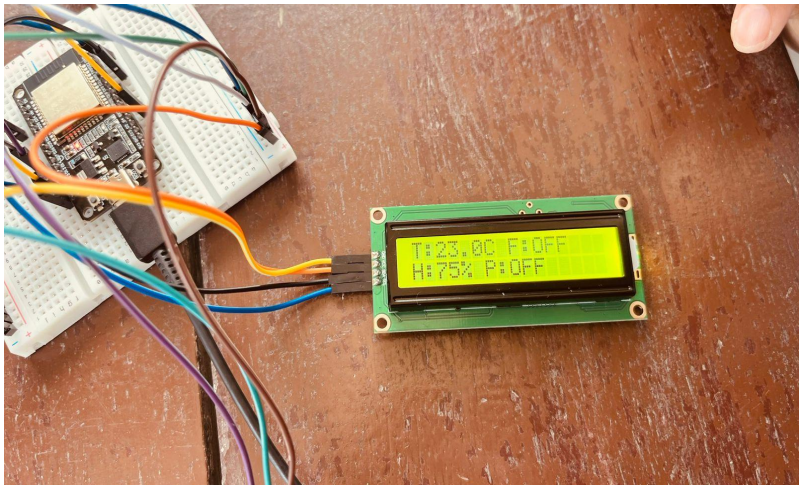


Figure 38: temperature raised

4.9 Test no.9 - Change in Humidity

Table 9: Change in humidity

Objectives	Change humidity.
Action	Blow air to the DHT22 sensor.
Expected results	The sensor shows new value.
Actual results	The sensor showed new value.
conclusion	The test was successful.

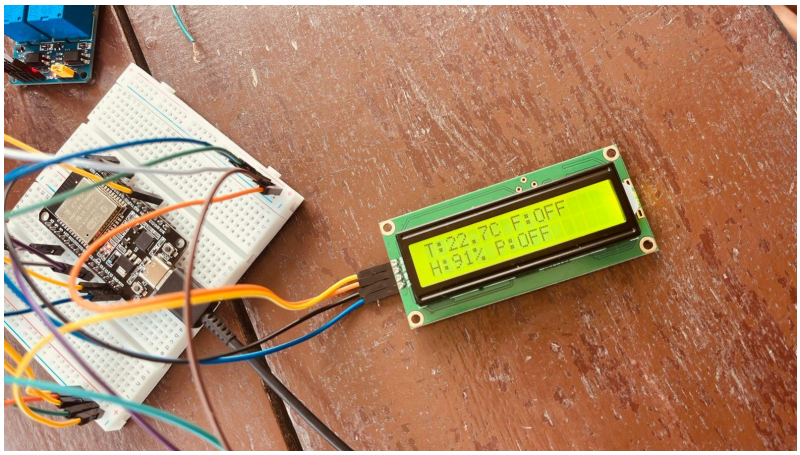


Figure 39: change in humidity

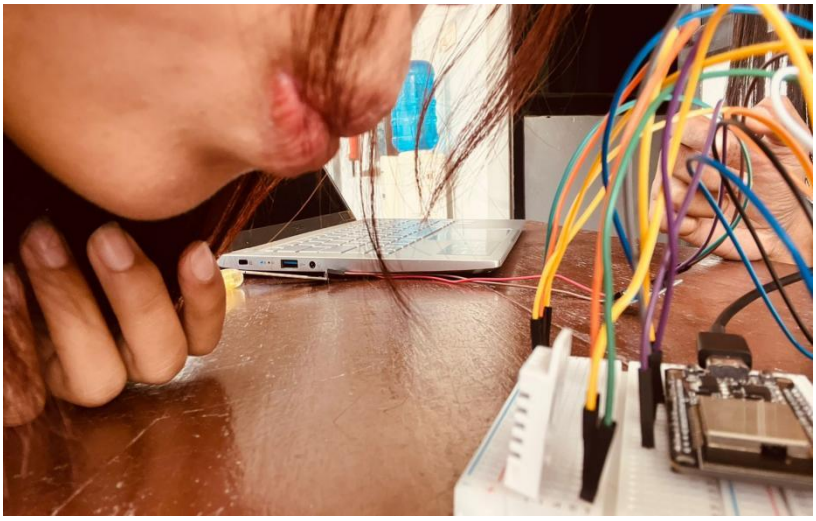


Figure 40: blow in DHT22 to change humidity

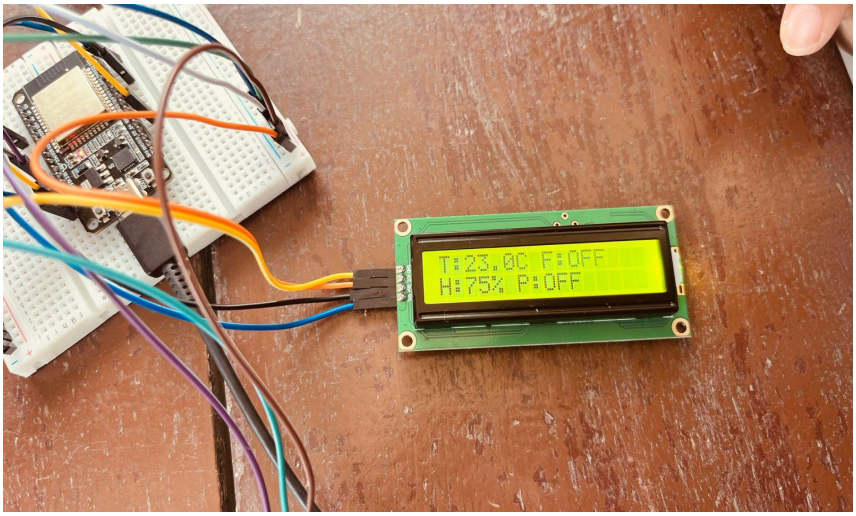


Figure 41: humidity decreased

4.10 Test no.10 - Relay click test

Table 10: Relay click test

Objectives	To test click sound on relay module
Action	Connect relay module to battery and esp32 and control system is applied to turn the relay ON and OFF.
Expected results	Relay produces click sound when activated and deactivated.
Actual results	Relay produces click sound when activated and deactivated.
conclusion	The test was successful.

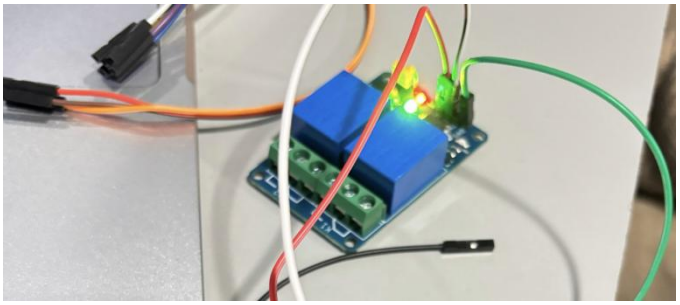


Figure 42: when relay is activated

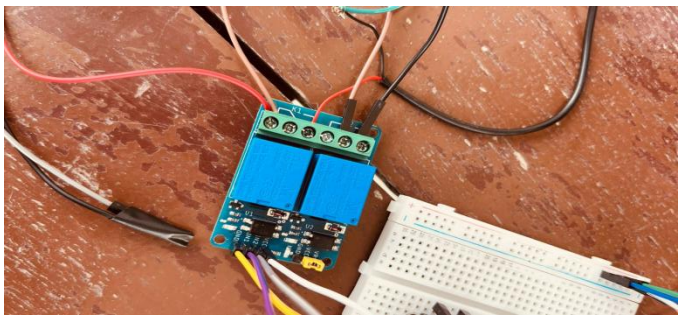


Figure 43: when relay is deactivated

Appendix

5. Future work

The existing Greenhouse Monitoring and Control System is a functional Model and it is possible to make some Enhancements when developed in the future. Appendix

6. Conclusion

In conclusion, the smart greenhouse helps to improve the growth of plants in easier and improved ways with the help of IOT. While working on this project, the temperature, light, soil moisture, and lights were continuously checked and measured using different devices like sensors. Appendix

7. References

Thonny, Python IDE for beginners, https://thonny.org/. Accessed 16 January 2026.

draw.io, https://www.drawio.com/. Accessed 16 January 2026.

Node-RED: Low-code programming for event-driven applications, https://nodered.org/. Accessed 16 January 2026.

Cirkit Designer IDE, https://app.cirkitdesigner.com/. Accessed 16 January 2026.

"Components of IOT and relation with Cloud Computing." *GeeksforGeeks*, 12 July 2025, https://www.geeksforgeeks.org/blogs/components-of-iot-and-relation-with-cloud-computing/. Accessed 16 January 2026.

"ESP32 for IoT: A Complete Guide." *Nabto*, https://www.nabto.com/guide-to-iot-esp-32/. Accessed 16 January 2026.

"Introduction to Internet of Things - IOT." *GeeksforGeeks*, 17 October 2025, https://www.geeksforgeeks.org/computer-networks/introduction-to-internet-of-things-iot-set-1/. Accessed 16 January 2026.

Lanyon, Charles. "Greenhouse." *Wikipedia*, https://en.wikipedia.org/wiki/Greenhouse. Accessed 16 January 2026.

8. Appendix

8.1. Appendix A: Source code

8.1.1. main.py

```
import dht
import network
import ujson
import time
from machine import Pin, ADC
from umqtt.simple import MQTTClient

# -------- DHT SENSOR --------
dht_sensor = dht.DHT22(Pin(4))

# -------- WIFI --------
Wifi_ssid = "IIC_WIFI"
Wifi_pass = "!tah@rIntl2025"

# -------- MQTT --------
MQTT_CLIENT_ID = "290389975a754872a92f77eb36aed48e"
MQTT_BROKER = "290389975a754872a92f77eb36aed48e.s1.eu.hivemq.cloud"
MQTT_PORT = 8883
MQTT_SSL = True
MQTT_USER = "shradhalimbu"
MQTT_PASS = "Shradha@123"
MQTT_SQT_PARAMS = {"server_hostname": MQTT_BROKER}

MQTT_TOPIC1 = "IOT/DHT"
MQTT_TOPIC2 = "IOT/MOTOR"
MQTT_TOPIC3 = "IOT/PUMP"
```

```
# -------- LCD --------
from machine import SoftI2C
from machine_i2c_lcd import I2cLcd

I2C_ADDR = 0x27
I2C_NUM_ROWS = 2
I2C_NUM_COLS = 16

i2c = SoftI2C(sda=Pin(21), scl=Pin(22), freq=100000)
lcd = I2cLcd(i2c, I2C_ADDR, I2C_NUM_ROWS, I2C_NUM_COLS)

# -------- SENSORS & ACTUATORS --------
SOIL_PIN = 34
PUMP_PIN = 27
FAN_PIN = 26

soil_sensor = ADC(Pin(SOIL_PIN))
soil_sensor.atten(ADC.ATTN_11DB)

pump = Pin(PUMP_PIN, Pin.OUT)
fan = Pin(FAN_PIN, Pin.OUT)

# -------- WIFI CONNECT --------
def connect_wifi():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    wlan.connect(Wifi_ssid, Wifi_pass)
    while not wlan.isconnected():
        time.sleep(0.5)
    print("WiFi connected")
```

```
# -------- MQTT CONNECT --------
def connect_mqtt():
    client = MQTTClient(
        MQTT_CLIENT_ID,
        MQTT_BROKER,
        port=MQTT_PORT,
        ssl=MQTT_SSL,
        user=MQTT_USER,
        password=MQTT_PASS,
        ssl_params=MQTT_SQT_PARAMS
    )
    client.connect()
    return client

def set_cb(topic, msg):
    print(topic.decode(), msg.decode())

# -------- SETUP --------
connect_wifi()
client = connect_mqtt()
client.set_callback(set_cb)
client.subscribe(MQTT_TOPIC1)
client.subscribe(MQTT_TOPIC2)
client.subscribe(MQTT_TOPIC3)

# -------- LOOP --------
while True:
    try:
        # Read DHT
        dht_sensor.measure()
```

```
temp = dht_sensor.temperature()
hum = dht_sensor.humidity()

# Read soil moisture
soil_value = soil_sensor.read()
print("Soil Moisture Value:", soil_value)

# -------- FAN LOGIC --------
if temp <  24:
    fan.value(1)         # Fan ON
    fan_state = "OFF"
else:
    fan.value(0)         # Fan OFF
    fan_state = "ON"

print("Fan Relay:", fan_state)

# -------- PUMP LOGIC --------
if soil_value < 2400:     # Dry soil
    pump.value(0)        # Pump ON
    pump_state = "ON"
    client.publish(MQTT_TOPIC3, "ON")
else:
    pump.value(1)        # Pump OFF
    pump_state = "OFF"
    client.publish(MQTT_TOPIC3, "OFF")

print("Pump Relay:", pump_state)

# -------- LCD DISPLAY --------
lcd.clear()
```

```
        lcd.move_to(0, 0)
        lcd.putstr("T:{:.1f}C F:{}".format(temp, fan_state))

        lcd.move_to(0, 1)
        lcd.putstr("H:{}% P:{}".format(int(hum), pump_state))

        # -------- MQTT DATA --------
        client.publish(
            MQTT_TOPIC1,
            ujson.dumps({
                "temperature": temp,
                "humidity": hum,
                "soil": soil_value
            })
        )

        client.check_msg()

    except Exception as e:
        print("Error:", e)

    time.sleep(5)
```

8.1.2. Lcd_api.py

```
    # Provides an API for talking to HD44780 compatible character LCDs.
    # https://github.com/dhylands/python_lcd/tree/master/lcd
    import time

    class LcdApi:
```

```
"""Implements the API for talking with HD44780 compatible character LCDs.
This class only knows what commands to send to the LCD, and not how to get
them to the LCD.

It is expected that a derived class will implement the hal_xxx functions.
"""

# The following constant names were lifted from the avrlib lcd.h
# header file, however, I changed the definitions from bit numbers
# to bit masks.
#
# HD44780 LCD controller command set

LCD_CLR = 0x01              # DB0: clear display
LCD_HOME = 0x02             # DB1: return to home position

LCD_ENTRY_MODE = 0x04       # DB2: set entry mode
LCD_ENTRY_INC = 0x02        # --DB1: increment
LCD_ENTRY_SHIFT = 0x01      # --DB0: shift

LCD_ON_CTRL = 0x08          # DB3: turn lcd/cursor on
LCD_ON_DISPLAY = 0x04       # --DB2: turn display on
LCD_ON_CURSOR = 0x02        # --DB1: turn cursor on
LCD_ON_BLINK = 0x01         # --DB0: blinking cursor

LCD_MOVE = 0x10             # DB4: move cursor/display
LCD_MOVE_DISP = 0x08        # --DB3: move display (0-> move cursor)
LCD_MOVE_RIGHT = 0x04       # --DB2: move right (0-> left)

LCD_FUNCTION = 0x20         # DB5: function set
LCD_FUNCTION_8BIT = 0x10    # --DB4: set 8BIT mode (0->4BIT mode)
```

```
    LCD_FUNCTION_2LINES = 0x08  # --DB3: two lines (0->one line)
    LCD_FUNCTION_10DOTS = 0x04  # --DB2: 5x10 font (0->5x7 font)
    LCD_FUNCTION_RESET = 0x30   # See "Initializing by Instruction" section

    LCD_CGRAM = 0x40          # DB6: set CG RAM address
    LCD_DDRAM = 0x80          # DB7: set DD RAM address

    LCD_RS_CMD = 0
    LCD_RS_DATA = 1

    LCD_RW_WRITE = 0
    LCD_RW_READ = 1

    def __init__(self, num_lines, num_columns):
        self.num_lines = num_lines
        if self.num_lines > 4:
            self.num_lines = 4
        self.num_columns = num_columns
        if self.num_columns > 40:
            self.num_columns = 40
        self.cursor_x = 0
        self.cursor_y = 0
        self.implied_newline = False
        self.backlight = True
        self.display_off()
        self.backlight_on()
        self.clear()
        self.hal_write_command(self.LCD_ENTRY_MODE | self.LCD_ENTRY_INC)
        self.hide_cursor()
        self.display_on()
```

```
    def clear(self):
        """Clears the LCD display and moves the cursor to the top left
        corner.
        """
        self.hal_write_command(self.LCD_CLR)
        self.hal_write_command(self.LCD_HOME)
        self.cursor_x = 0
        self.cursor_y = 0

    def show_cursor(self):
        """Causes the cursor to be made visible."""
        self.hal_write_command(self.LCD_ON_CTRL | self.LCD_ON_DISPLAY |
                               self.LCD_ON_CURSOR)

    def hide_cursor(self):
        """Causes the cursor to be hidden."""
        self.hal_write_command(self.LCD_ON_CTRL | self.LCD_ON_DISPLAY)

    def blink_cursor_on(self):
        """Turns on the cursor, and makes it blink."""
        self.hal_write_command(self.LCD_ON_CTRL | self.LCD_ON_DISPLAY |
                               self.LCD_ON_CURSOR | self.LCD_ON_BLINK)

    def blink_cursor_off(self):
        """Turns on the cursor, and makes it no blink (i.e. be solid)."""
        self.hal_write_command(self.LCD_ON_CTRL | self.LCD_ON_DISPLAY |
                               self.LCD_ON_CURSOR)

    def display_on(self):
        """Turns on (i.e. unblanks) the LCD."""
        self.hal_write_command(self.LCD_ON_CTRL | self.LCD_ON_DISPLAY)
```

```
    def display_off(self):
        """Turns off (i.e. blanks) the LCD."""
        self.hal_write_command(self.LCD_ON_CTRL)

    def backlight_on(self):
        """Turns the backlight on.

        This isn't really an LCD command, but some modules have backlight
        controls, so this allows the hal to pass through the command.
        """
        self.backlight = True
        self.hal_backlight_on()

    def backlight_off(self):
        """Turns the backlight off.

        This isn't really an LCD command, but some modules have backlight
        controls, so this allows the hal to pass through the command.
        """
        self.backlight = False
        self.hal_backlight_off()

    def move_to(self, cursor_x, cursor_y):
        """Moves the cursor position to the indicated position. The cursor
        position is zero based (i.e. cursor_x == 0 indicates first column).
        """
        self.cursor_x = cursor_x
        self.cursor_y = cursor_y
        addr = cursor_x & 0x3f
        if cursor_y & 1:
```

```
            addr += 0x40    # Lines 1 & 3 add 0x40
        if cursor_y & 2:    # Lines 2 & 3 add number of columns
            addr += self.num_columns
        self.hal_write_command(self.LCD_DDRAM | addr)

    def putchar(self, char):
        """Writes the indicated character to the LCD at the current cursor
        position, and advances the cursor by one position.
        """
        if char == '\n':
            if self.implied_newline:
                # self.implied_newline means we advanced due to a wraparound,
                # so if we get a newline right after that we ignore it.
                self.implied_newline = False
            else:
                self.cursor_x = self.num_columns
        else:
            self.hal_write_data(ord(char))
            self.cursor_x += 1
        if self.cursor_x >= self.num_columns:
            self.cursor_x = 0
            self.cursor_y += 1
            self.implied_newline = (char != '\n')
        if self.cursor_y >= self.num_lines:
            self.cursor_y = 0
        self.move_to(self.cursor_x, self.cursor_y)

    def putstr(self, string):
        """Write the indicated string to the LCD at the current cursor
        position and advances the cursor position appropriately.
        """
```

```
        for char in string:
            self.putchar(char)

    def custom_char(self, location, charmap):
        """Write a character to one of the 8 CGRAM locations, available
        as chr(0) through chr(7).
        """
        location &= 0x7
        self.hal_write_command(self.LCD_CGRAM | (location << 3))
        self.hal_sleep_us(40)
        for i in range(8):
            self.hal_write_data(charmap[i])
            self.hal_sleep_us(40)
        self.move_to(self.cursor_x, self.cursor_y)

    def hal_backlight_on(self):
        """Allows the hal layer to turn the backlight on.

        If desired, a derived HAL class will implement this function.
        """
        pass

    def hal_backlight_off(self):
        """Allows the hal layer to turn the backlight off.

        If desired, a derived HAL class will implement this function.
        """
        pass

    def hal_write_command(self, cmd):
        """Write a command to the LCD.
```

```
        It is expected that a derived HAL class will implement this
        function.
        """
        raise NotImplementedError

    def hal_write_data(self, data):
        """Write data to the LCD.

        It is expected that a derived HAL class will implement this
        function.
        """
        raise NotImplementedError

    # This is a default implementation of hal_sleep_us which is suitable
    # for most micropython implementations. For platforms which don't
    # support `time.sleep_us()` they should provide their own implementation
    # of hal_sleep_us in their hal layer and it will be used instead.
    def hal_sleep_us(self, usecs):
        """Sleep for some time (given in microseconds)."""
        time.sleep_us(usecs)  # NOTE this is not part of Standard Python library, specific hal layers will need to override this
```

8.1.3. Machine_i2c_lcd.py

```
# Implements a HD44780 character LCD connected via PCF8574 on I2C.
# This was tested with: https://www.wemos.cc/product/d1-mini.html
# https://github.com/dhylands/python_lcd/blob/master/lcd/machine_i2c_lcd.py

from lcd_api import LcdApi
from time import sleep_ms

# The PCF8574 has a jumper selectable address: 0x20 - 0x27
DEFAULT_I2C_ADDR = 0x27

# Defines shifts or masks for the various LCD line attached to the PCF8574

MASK_RS = 0x01
MASK_RW = 0x02
MASK_E = 0x04
SHIFT_BACKLIGHT = 3
SHIFT_DATA = 4



class I2cLcd(LcdApi):
    """Implements a HD44780 character LCD connected via PCF8574 on I2C."""

    def __init__(self, i2c, i2c_addr, num_lines, num_columns):
        self.i2c = i2c
        self.i2c_addr = i2c_addr
        self.i2c.writeto(self.i2c_addr, bytearray([0]))
        sleep_ms(20)   # Allow LCD time to powerup
        # Send reset 3 times
        self.hal_write_init_nibble(self.LCD_FUNCTION_RESET)
        sleep_ms(5)    # need to delay at least 4.1 msec
```

```
        self.hal_write_init_nibble(self.LCD_FUNCTION_RESET)
        sleep_ms(1)
        self.hal_write_init_nibble(self.LCD_FUNCTION_RESET)
        sleep_ms(1)
        # Put LCD into 4 bit mode
        self.hal_write_init_nibble(self.LCD_FUNCTION)
        sleep_ms(1)
        LcdApi.__init__(self, num_lines, num_columns)
        cmd = self.LCD_FUNCTION
        if num_lines > 1:
            cmd |= self.LCD_FUNCTION_2LINES
        self.hal_write_command(cmd)

    def hal_write_init_nibble(self, nibble):
        """Writes an initialization nibble to the LCD.

        This particular function is only used during initialization.
        """
        byte = ((nibble >> 4) & 0x0f) << SHIFT_DATA
        self.i2c.writeto(self.i2c_addr, bytearray([byte | MASK_E]))
        self.i2c.writeto(self.i2c_addr, bytearray([byte]))

    def hal_backlight_on(self):
        """Allows the hal layer to turn the backlight on."""
        self.i2c.writeto(self.i2c_addr, bytearray([1 << SHIFT_BACKLIGHT]))

    def hal_backlight_off(self):
        """Allows the hal layer to turn the backlight off."""
        self.i2c.writeto(self.i2c_addr, bytearray([0]))

    def hal_write_command(self, cmd):
```

```
        """Writes a command to the LCD.

        Data is latched on the falling edge of E.
        """
        byte = ((self.backlight << SHIFT_BACKLIGHT) | (((cmd >> 4) & 0x0f) << SHIFT_DATA))
        self.i2c.writeto(self.i2c_addr, bytearray([byte | MASK_E]))
        self.i2c.writeto(self.i2c_addr, bytearray([byte]))
        byte = ((self.backlight << SHIFT_BACKLIGHT) | ((cmd & 0x0f) << SHIFT_DATA))
        self.i2c.writeto(self.i2c_addr, bytearray([byte | MASK_E]))
        self.i2c.writeto(self.i2c_addr, bytearray([byte]))
        if cmd <= 3:
            # The home and clear commands require a worst case delay of 4.1 msec
            sleep_ms(5)

    def hal_write_data(self, data):
        """Write data to the LCD."""
        byte = (MASK_RS | (self.backlight << SHIFT_BACKLIGHT) | (((data >> 4) & 0x0f) << SHIFT_DATA))
        self.i2c.writeto(self.i2c_addr, bytearray([byte | MASK_E]))
        self.i2c.writeto(self.i2c_addr, bytearray([byte]))
        byte = (MASK_RS | (self.backlight << SHIFT_BACKLIGHT) | ((data & 0x0f) << SHIFT_DATA))
        self.i2c.writeto(self.i2c_addr, bytearray([byte | MASK_E]))
        self.i2c.writeto(self.i2c_addr, bytearray([byte]))
```

8.1.4. Lcd_test.py

```
# Rui Santos & Sara Santos - Random Nerd Tutorials
# Complete project details at https://RandomNerdTutorials.com/micropython-i2c-lcd-esp32-esp8266/

# ya dekhi
from machine import Pin, SoftI2C
from machine_i2c_lcd import I2cLcd
from time import sleep

# Define the LCD I2C address and dimensions
I2C_ADDR = 0x27
I2C_NUM_ROWS = 2
I2C_NUM_COLS = 16

# Initialize I2C and LCD objects
i2c = SoftI2C(sda=Pin(21), scl=Pin(22), freq=400000)

# for ESP8266, uncomment the following line
#i2c = SoftI2C(sda=Pin(4), scl=Pin(5), freq=400000)

lcd = I2cLcd(i2c, I2C_ADDR, I2C_NUM_ROWS, I2C_NUM_COLS)

# ya samma

lcd.putstr("It's working :)")
sleep(4)

try:
    while True:
        # Clear the LCD
```

```
        lcd.clear()
        # Display two different messages on different lines
        # By default, it will start at (0,0) if the display is empty
        lcd.putstr("Hello World!")t
        sleep(2)
        lcd.clear()
        # Starting at the second line (0, 1)
        lcd.move_to(0, 1)
        lcd.putstr("Hello World!")
        sleep(2)

except KeyboardInterrupt:
    # Turn off the display
    print("Keyboard interrupt")
    lcd.backlight_off()
lcd.display_off()
```

8.2. Appendix B: Testing & Supporting Materials

2.2. system overview

The temperature, humidity, and soil moisture will be checked in the greenhouse all the time using digital sensors. The system will instantly turn on the fan to ventilate the air in the system when the temperature or humidity goes beyond the preset level, and the water is kept in bottles as a water container. Similarly, the motor pump will start working if the soil moisture sensor records low moisture levels. The records will be shown instantly on MQTT. The dashboard will let the users to See the live environmental and Check system condition.

Plants will be placed both aesthetically and functionally. The greenhouse frame is going to be covered with plastic to maintain the heat and keep the humidity, and the plants will be set in a greenhouse.

2.2.1 Block Diagrams

The sensor readings are sent to the micro-controller ESP32 that provides the central processing unit. The ESP32 carries out the received data, provides real-time data on an LCD, transmits data to the cloud where it undergoes remote monitoring and measures control signals to be transmitted to operate the external devices using a relay module. The system actuators are supplied with power by a battery which makes them reliable.

The sensors work on the principle of constant monitoring of the environment and the soil state and transmitting this information to the ESP32 microcontroller. The ESP32 determines the necessary actions depending on the threshold values which are used in one of the programs According to the threshold values used in one of the programs, the ESP32 can determine whether or not actions should be taken, e.g., the relay module must be turned on. The switch to turn the water pump and DC motor ON or OFF is configured by the relay to be used in irrigation or mechanical work. Simultaneously,

sensor information is shown in the LCD to monitor it locally and sent to the cloud to obtain it remotely. All components are interconnected on the half breadboard which enables the flow of signals and system integration to increase efficiency and make the control system automated and smart.

2.2.2 Flowchart

They also enhance communication because they give a clear visual description of the interaction of various components in a system.

The Smart Greenhouse IOT system flowchart demonstrates the overall workflow of the greenhouse, beginning with the activation of the system and proceeding to its constant monitoring and automated control. The system process begins with the ESP32 Node Red being turned on this is the part that controls the Smart Greenhouse IOT system. All the related parts, including sensors, the LCD display, Battery and the communication protocols are also turned on with the ESP32 to make sure that the system is well prepared. After the process of initializing is finished, the system reads the information of the attached sensors. These are the DHT22 temperature and humidity sensor and the soil moisture sensor. At this level, the system checks the validity of sensor data and that it is within acceptable range. When one of the sensors does not respond or provides invalid data, a sensor error will be detected by the system and make sure that no wrong decision is taken on the basis of invalid data.

The system is able to process a sensor reading at a time when valid data exists. DHT22 sensor is used to offer the temperature and humidity readings in the greenhouse. In case the temperature or humidity levels are high compared to the set upper limit, the relay module triggers the cooling fan automatically to control the internal conditions. When the numbers are in the safe range, the fan will be off not to waste energy.

The water in the soil is being tested by the soil moisture sensor at all times. If the soil has water than it should the ESP32 turns on the relay and this starts the water pump

that gives water to the plants. When the soil has water again the water pump is turned off by itself.

The system also provides manual control functionality. Through a cloud based dashboard on the internet the user can override the decisions and control things, like the fan or water pump itself. Each time some manual command is given, the ESP32 will change the state of the actuators in order to keep the behavior of the system and the input of the user in phase. During the operation process, the sensor reading and actuator status are sent to the cloud via the MQTT communication protocol. This information is presented on a Web and can additionally be indicated locally on the LCD display and can give real-time feedback to the user. The system constantly examines whether the sensor value is within any of the safety limits that have been set. The system will send an alert or a message to the user when the conditions are dangerous.

Finally, once each cycle of the data Obtaining, decision-making, and control has been complete, a short delay period then follows after which the system is set to repeat itself. Such a looping process allows real-time monitoring and automation of the process guaranteeing that the greenhouse environment is always optimal to facilitate growth of plants.

2.2.3. Circuits diagram

There is an I2C LCD showing real time values of soil moisture level, temperature and humidity, and system status. One of the devices an electrically controlled switch is applied as a relay module to allow the low-power ESP32 to operate a more powerful device, namely a water pump with an external battery in this case. In case the soil becomes excessively dry the ESP32 transmits a signal to the relay to become ON, whereas upon detecting adequate moisture it switches the pump OFF.

The soil moisture sensor operates continuously with analog/digital signals broad casted in response to the amount of moisture in the soil to the ESP32 and the DHT sensor

gives the temperature in the environment and the humidity. The ESP32 process these readings and displays them on the LCD so that one can easily monitor them. In case the moisture content of the soil drops below a predetermined value, the ESP32 will trigger the relay module, and the external battery will be connected to the relay module, which will turn on the water pump and water can flow to the plants. When a sufficient equilibrium is established in the moisture content of soil, the ESP32 will change switching off the pump by communicating with the relay. This forms a self-controlling automated network which saves water, man labor, and offers optimum irrigation to the plants.

2.2.4 Schematics Diagram

It assigns the symbols to label the elements of resistor, sensor, micro controller, power supply, and output device. A schematic does not show the physical location of components, but instead it focuses on the logic of the connections to make the users enjoy how the flow of current would be and that the signals would be transmitted by the system.

The major purpose of a schematic diagram is to ease the design of the circuit in terms of comprehension, analysis and troubleshooting. It is an engineering rule applicable to engineers and students before building the hardware. One of its applications can also be ensuring that the components are well interconnected and the system would function as intended and therefore is a mandatory step in the design and documentation of electronics.

The smart monitoring and control system is represented as a scheme diagram below and is based on ESP32. The data on the DHT22 temperature and humidity sensor and the soil moisture sensor is supplied to ESP32 Node-Red that serves as the main controller. Such sensors provide the environment information to the ESP32 which is processed by the ESP32. I2C LCD display is connected directly to the ESP32 and configured through the SDA and SCL pins and is used to display the real-time values that include temperature, humidity and soil moisture. Breadboard power rails are then employed to supply power to the entire components via a 5V supply.

It has also a 2-channel relay module installed to make the ESP32 be able to control more powerful devices in a safe way. The relay is attached to two DC motors one being a fan and the other a water pump. After the soil dries, the ESP32 will turn on the water pump by switching the relay and the fan as well will be turned on when the temperature increases. This has made the system automatic and efficient that can be applied in smart agriculture or greenhouse monitoring.

2.3.2. Software components

i. **Thonny IDE:** Thonny is a programming and configuration tool of ESP32 micro-controller. It is a program that people use to make, send and fix programs, for the ESP32 micro-controller. It is compatible with Micro-python used to create easy and quick development of IOT applications. It also provides serial monitor which allows viewing sensor data and system messages in real-time. This assists in checking proper functionality of systems and detection of program errors.

Figure 44: Thonny IDE logo

ii. **MQTT Protocol:** The MQTT is a lightweight messaging protocol, which support low-bandwidth IOT devices. The MQTT enables the ESP32 and the cloud platform to exchange the data in an efficient and reliable manner. In this project, MQTT is applied to send sensor data and get control commands to be able to monitor the greenhouse system remotely and automate it. This means we can check on the greenhouse and control the greenhouse from far away too.

iii. **Node-RED:** Node-RED is a flow-based programming language that is applied in connecting IOT devices, data processing, and visualization. It is compatible with the MQTT protocol and commonly used in the IOT. It reads the sensors of the MQTT and inserts the data of these sensors into a dashboard that can be easily interpreted by people. The use of Node-RED enables real-time monitoring of greenhouse conditions and remote control of the devices, such as the water pump and the motor fan, with the help of Node-RED and make the system more flexible and easier to use.

iv. **Serial Monitor:** It is used to send or receive information from by the ESP32 such as temperature, humidity, and soil moisture values. It is also useful in debugging the program because it demonstrates whether the ESP32 is being able to read sensor data properly. Serial output can be used to monitor that a system is working correctly and that control logic is executing as expected through testing.

3.1 planning and Design

The primary goal was to create an automated process that would be able to measure temperature, humidity, and soil moisture and regulate the irrigation systems in order to keep the greenhouse in Suitable conditions

Under the Initial planning phase, a system requirement analysis was made to identify functional as well as non-functional requirements. The system was technologically programmed to continuously check on the conditions of the environment, automatically turn the water pump on or off according to the soil moisture, operate high-power Tools under the influence of relay modules, and show sensor information in real time on an LCD. Additional non-functional factors were the reliability, reduced power consumption, electrical safety, and simplicity of the system based on a modular structure.

In agreement with these requirements, the relevant hardware components were chosen. ESP32 micro-controller has been selected as the key controller because it has a high performance, many GPIO pins, and runs Micro-python with Thonny IDE library. Temperature and humidity were monitored using a DHT22 sensor, and a soil moisture sensor was used to have automated supplying water The relay modules were implemented to preventative actuate pumps and motors, an LCD was implemented to show system status and sensor values.

The circuit diagram was drawn in details to be sure of proper connections, proper distribution of power and a common ground where the circuit would operate properly. Flexible testing and verification Individual component verification was carried out through breadboard prototyping and then the system was completely integrated. The system logic was executed as per a flowchart, sensor data was being read continuously and control decisions were being made automatically using predefined thresholds.

Micro-Python was used in Thonny IDE to develop software, which could be easily coded, tested, and debugged. Integration and modular testing made sure that each component was operating properly to the point of integrating the entire system together. At the end

of the 16-day time, the fully operational prototype was accomplished, which proved to be a solid automated monitoring and control. Such design also gives an opportunity to extend it in the future with things like IOT connectivity and data logging.

3.2 Resource Allocations

he resources needed to develop the IOT Smart Greenhouse project include different hardware parts needed to sense, process, control, supply power, and interact with users. The main processing unit in the project was the ESP32 micro-controller that read sensor data, run control logic and made wireless communication possible. The DHT22 temperature and humidity sensor and the soil moisture sensor were used to monitor the environmental conditions and the sensors gave precise measurements that should be maintained to ensure that plant growth was done under the appropriate environmental conditions. A relay module was employed to safely connect the ESP32 to components with high power like the water pump and the motor fan. Water pump was used to automatically irrigate water when the moisture levels of soil were low and the motor fan was used to control the temperature and airflow in the greenhouse. The interaction with the users and local monitoring was facilitated by an LCD display that displayed real-time data. Components used to support the circuit included resistors to limit current and stabilize signals, jumper wires and half breadboard provided flexibility and temporary connections of the circuit when prototyping the system.

For software, programming, debugging and serial monitoring of ESP32 was done with Thonny IDE. The MQTT protocol was the means of communication between the greenhouse system and the monitoring platform, and the visualization sensor data and user-friendly dashboard of remote monitoring and control were performed with the help of the Node-RED.

The project hardware and materials were all bought in ABC Store, a local electronics dealer which is known to offer affordable and quality hardware components which can be used in IoT projects. ABC Store was selected because it has cost-effective and

reliable electronic parts that can be used in IoT applications. The quality elements and low prices combined helped obtain all the hardware needed to develop and implement the IoOA Smart Greenhouse project successfully.

3.3 System Development

3.3.1 For Temperature Monitoring:

- The ESP32 micro-controller was programmed to take the values of temperature at sensor after set intervals. appendix

Phase 2: Scanning temperature data

- The sensor kept on reading the temperature of the surrounding environment within the green house.
- The temperature data were captured and converted into a digital signal and transmitted to the micro-controller where it would be processed.

Phase 3: Decision are made through temperature

- The temperature level received was compared to a set point in the program.
- The microcontroller produced a control signal when the temperature was above the threshold.
- The signal was applied to the cooling fan other components by the relay module.

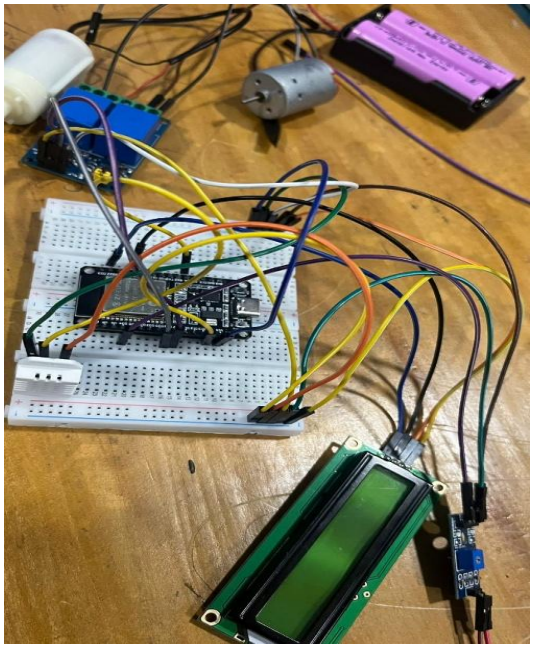


Figure 45: actual setup of temperature sensor

3.3.2. For Fan Control

- An external power supply was added in the form of a power supply to have sufficient power to the fan.

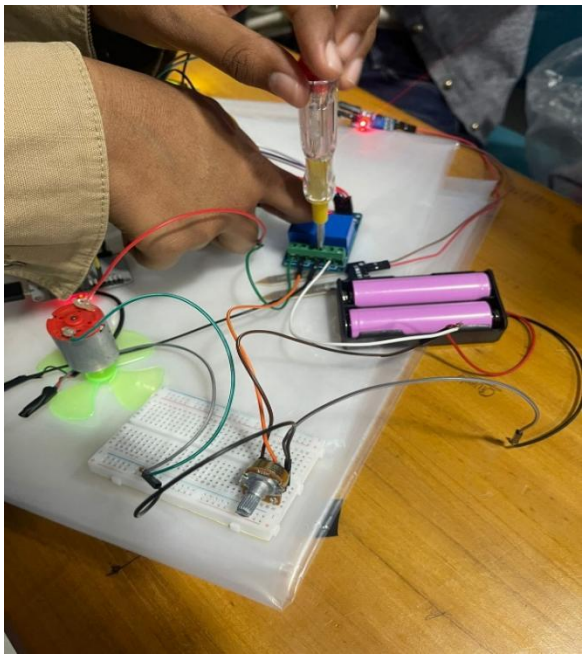


Figure 46 fan and relay module

Phase 2: Representation of program: Fan Control Logic

- The ESP32 micro-controller was programmed to respond to the relay values of temperature.
- The relay would be switched on when the temperature exceeded the temperature set point.
- When the relay was triggered, it enabled current to pass to the fan thus turning it on automatically.

Phase 3: Automatic Fan Work

- The fan was left ON when the temperature was higher than the threshold value.
- When the temperature level was brought back to a safe level, a relay was switched off.
- This guaranteed automatic ventilation within the green house without the need to do it manually.

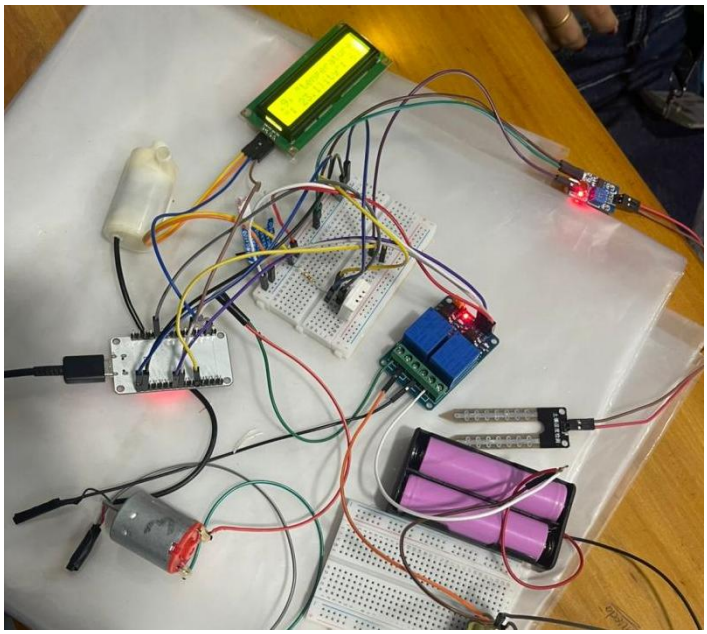


Figure 47 actual setup of fan development

3.3.3 For LCD Display

- The LCD was connected to the ESP32 micro-controller through LCD communication lines.
- The ground connections and power connections were well done at the right places to make it operate stably.

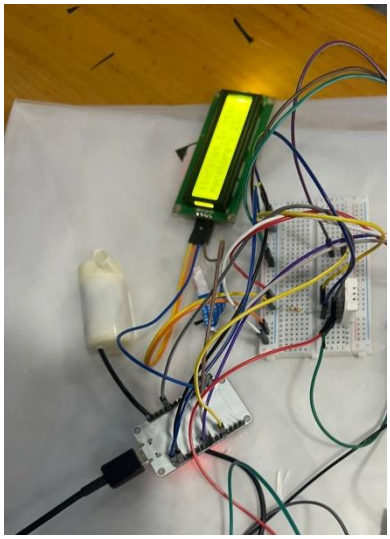


Figure 48 ESP32 and LCD connection

Phase 2: Presentation of Sensor Data.

- The data of temperature and system status of LCD is programmed by ESP32.
- The LCD screen showed real time temperature readings.
- The working condition of the equipment, e.g. the fan, was also displayed.

Phase 3: Display in Real Time.

- LCD displays the real time updated value frequently.
- The frequent change in values of temperature and fan is shown on LCD.
- LCD helps the user to monitor the greenhouse easily and without any big effort.

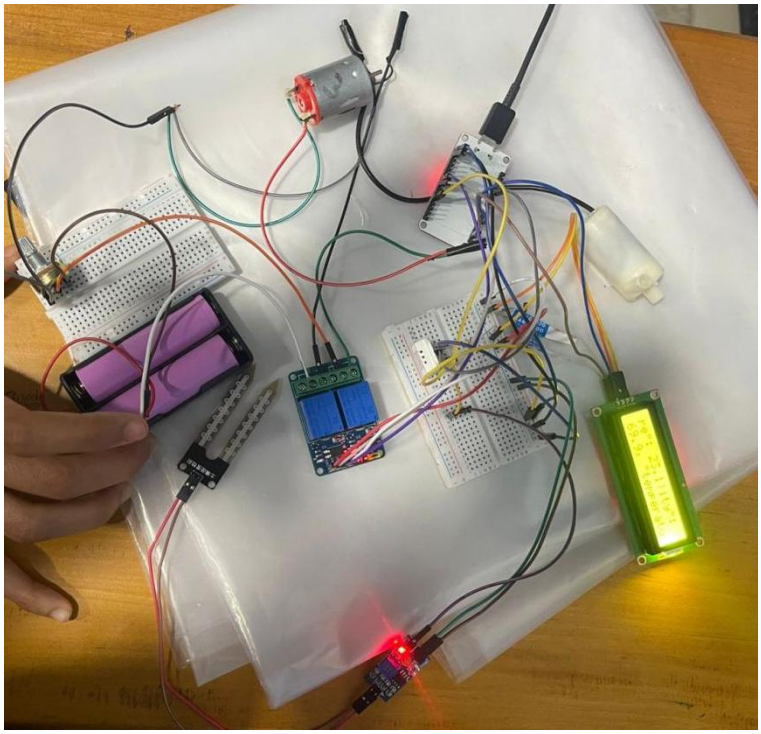


Figure 49 actual setup of LCD display

3.3.4 For Soil Moisture Monitoring

- Soil moisture sensor is attached to the breadboard with the power, GND and analog output pins using jumper wires.

Phase 2: Checking the value of soil

- Sensor is then placed in the soil to check the soil moisture content.
- Then the code is executed.
- With the help of executed code, the soil sensor helps to read the value of soil in the green house.

Phase 3: Decision are made through the moisture value of soil

- The logic for the soil is set to the program and was compared.
- Once the moisture of the greenhouse is below then the threshold level the control board helps to activate a signal to the relay module.

- Once the moisture of soil reaches to the level where the pump should be turned off, the pump turns off automatically.

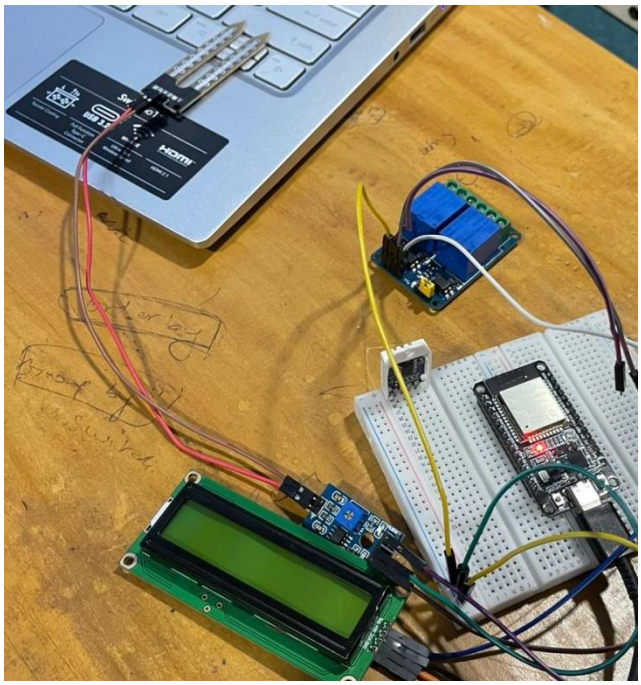


Figure 50: actual setup of soil sensor

3.3.5. For water pump

Phase 2: Water pump and Battery relation

- The (–) negative wire of water pump is connected with battery for the power to the pump.

Phase 3: Automatic Water pump work

- If the soil moisture value is less than threshold value, the water pump is turned on.
- The plant is watered with the help of water pump until the level of soil moisture meets the satisfied condition.
- Once the moisture level is satisfied, the control board sends a low signal at the relay.

- Water pump is then automatically turned off before the soil is over watered.

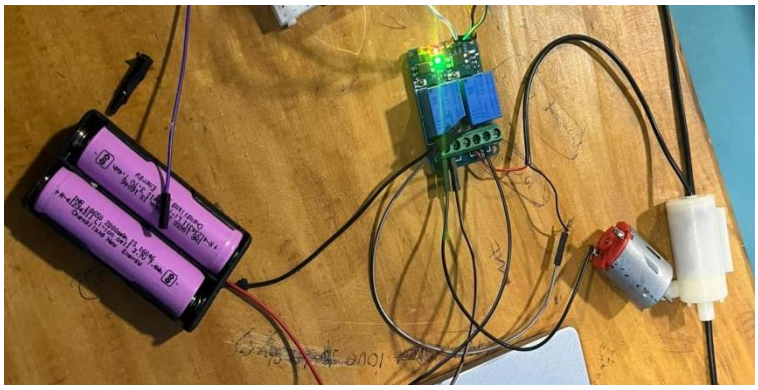


Figure 51: actual setup of water pump

Figure 52: Final smart green house

4. Result and Finding

4.11. Test no.11 - LCD values updates

Table 11: LCD values updates Testing

Objectives	Verify LCD updates display temperature, humidity and fan switch.
Action	Sensor is connected to the system and values are sent to LCD and when there is change in DHT22 sensor the LCD displays updated value with fan off or on condition.
Expected results	LCD display updates value.
Actual results	LCD displayed updated value.
conclusion	The test was successful.

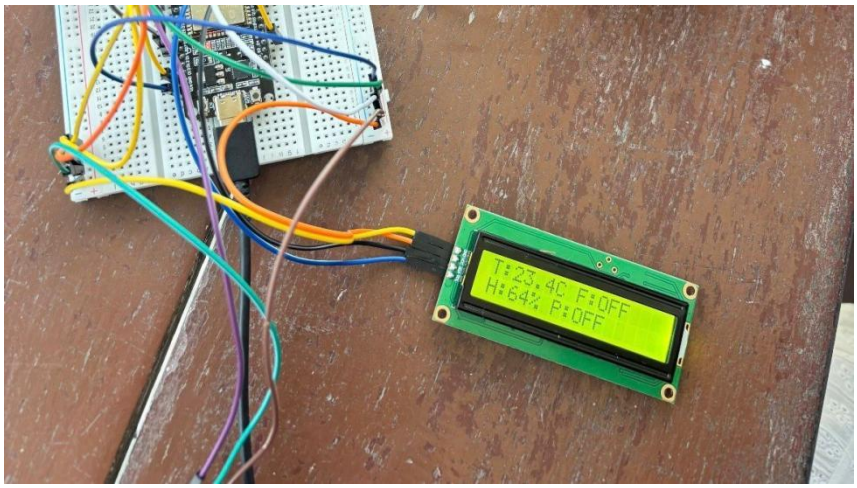


Figure 53: initial testing displayed on LCD

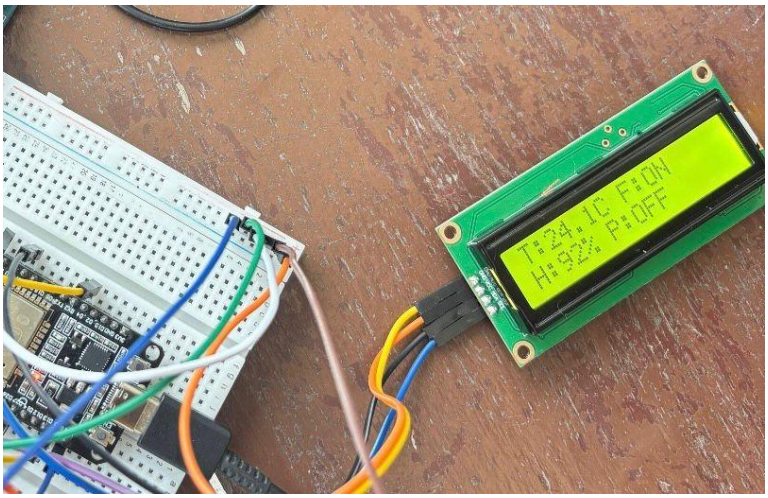


Figure 54: final testing displayed on LCD

4.12 Test no. 12–Unsuccessful pump on test

Table 12: Unsuccessful pump on test

Objectives	To test whether the motor pump automatically switch on when the value of the soil moisture sensor is lower than 2400 (dry soil).
Action	Insert the soil moisture sensor in dry soil and monitor the sensor value and check on pump behavior.
Expected results	In case the sensor value is low than 2400 that is when the soil is dry, motor pump should automatically switch on.
Actual results	The motor pump did not switch on when the soil was dry.
conclusion	The test was unsuccessful. The pump failed to respond accordingly and the system should be additionally checked and fixed.

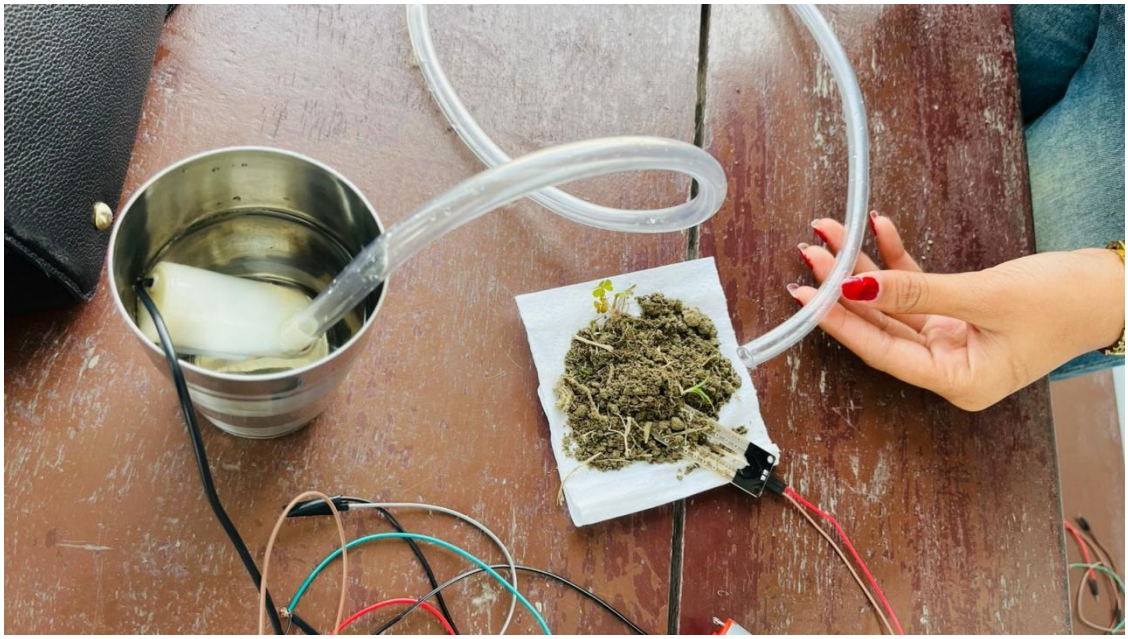


Figure 55: unsuccessful pump on test

4.13 Test no.13 – Successful pump on test

Table 13: Successful pump on test

Objectives	To check whether the motor pump starts running automatically when the soil moisture sensor value is below 2400 (dry soil condition).
Action	Put the sensor in the soil and wait and see the sensor reading and the actions of the pump in dry soil.
Expected results	When the sensor falls below 2400, dry soil should be detected and a motor pump automatically switched on.
Actual results	When the soil is dry, the motor pump switches on.
conclusion	The test was successful.

Figure 56: Successful pump on

4.14 Test no.14 – Unsuccessful pump off test

Table 14: Unsuccessful pump off test

Objectives	To test the motor pump whether it switches off automatically when the value of soil moisture sensor exceeds 2400 (wet soil condition).
Action	Wet the soil with water and insert the sensor in the wet soil and note sensor reading and pump operation.
Expected results	When the sensor goes above 2400, the wet soil should be detected and the motor pump should automatically switch off.
Actual results	The motor pump did not switch off even when the soil was wet.
conclusion	The test was unsuccessful. The system was not functioning exactly as it was expected and it needs additional debugging.

Figure 57: Unsuccessful pump off test

4.15 Test no.15 – Successful pump off test

Table 15: Successful pump off testing

Objectives	To determine whether the motor pump shuts itself when the value of soil moisture sensor exceeds 2400 (wet soil).
Action	Add water to soil and insert sensors of soil moisture into wet soil and monitor sensor reading and see the pump behavior.
Expected results	When the soil is wet, the system is expected to identify wet soil and the motor pump must be turned off automatically.
Actual results	The motor pump automatically turn off when the soil was wet.
conclusion	The test was successful.

Figure 58: sucessful moter test

4.16 Test no.16 –Unsuccessful fan on

Table 16: Unsuccessful fan on testing

Objectives	To check whether the cooling fan is activated automatically when the heat is up to 24C and above.
Action	Turn the heater to its highest level close to the DHT22 sensor until the LCD displays 24C or more then look at the fan behavior.
Expected results	At 24C or higher temperatures the system must pick up a high temperature and the cooling fan must switch on automatically.
Actual results	The cooling fan was not activated even when the temperature became 24C.

conclusion	The test was unsuccessful. The system failed to react accordingly and it needs additional debugging or corrective changes.

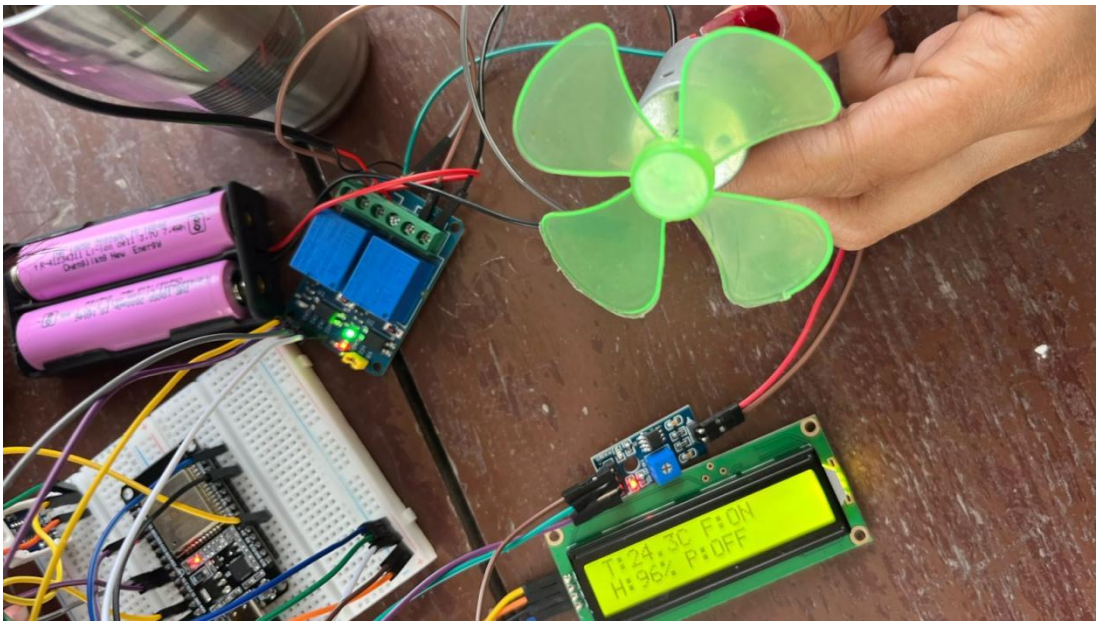


Figure 59: Unsuccessful fan on

4.17 Test no.17–Successful fan on test

Table 17: successful fan on testing

Objectives	To test the automatic on and off of the cooling fan when the temperature is 24C or above.
Action	Increase temperature at DHT22 sensor with the help of a lighter and monitor the behavior of the fan.
Expected results	When the temperature is 24C or higher than that, high temperature should be detected by the system and cool fan must go on automatically.
Actual results	The cooling fan was automatically switched on when the temperature is 24C.
conclusion	The test was successful.

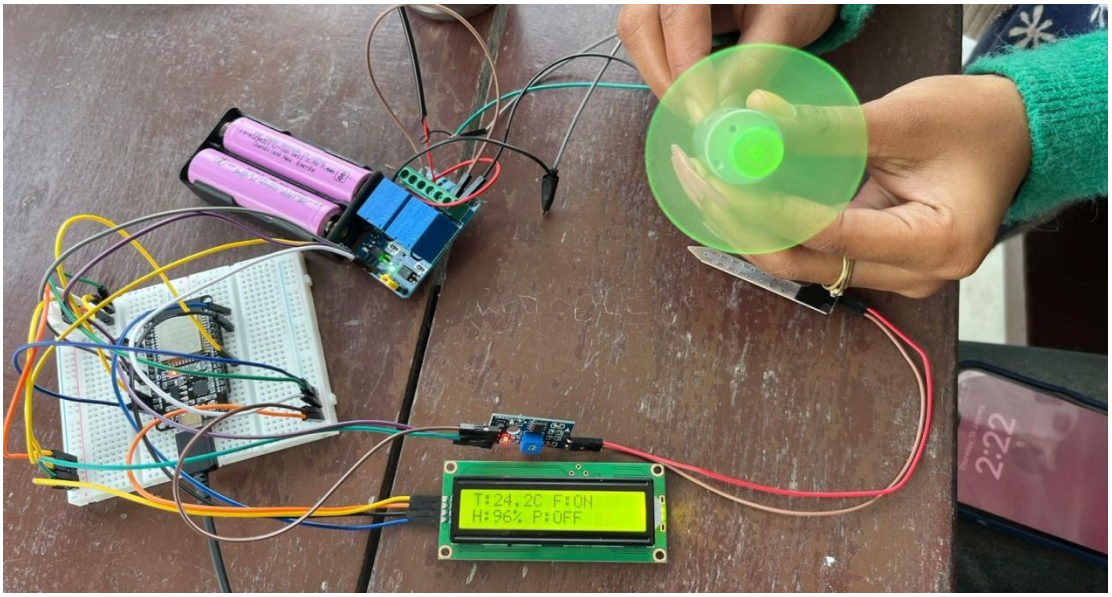


Figure 60: Successful fan test

4.18 Test no.18 – Successful fan off test

Table 18: Successful fan off testing

Objectives	To check whether the cooling fan is automatically turned off when the temperature is below 24C.
Action	Take the lighter away and let the temperature drop by itself then monitor the behavior of the fan.
Expected results	In cooling the temperature must go down to the normal range and the cooling fan must switch off automatically.

Actual results	When the temperature went back to the normal range, the cooling fan was automatically turned off.
conclusion	The test was successful.

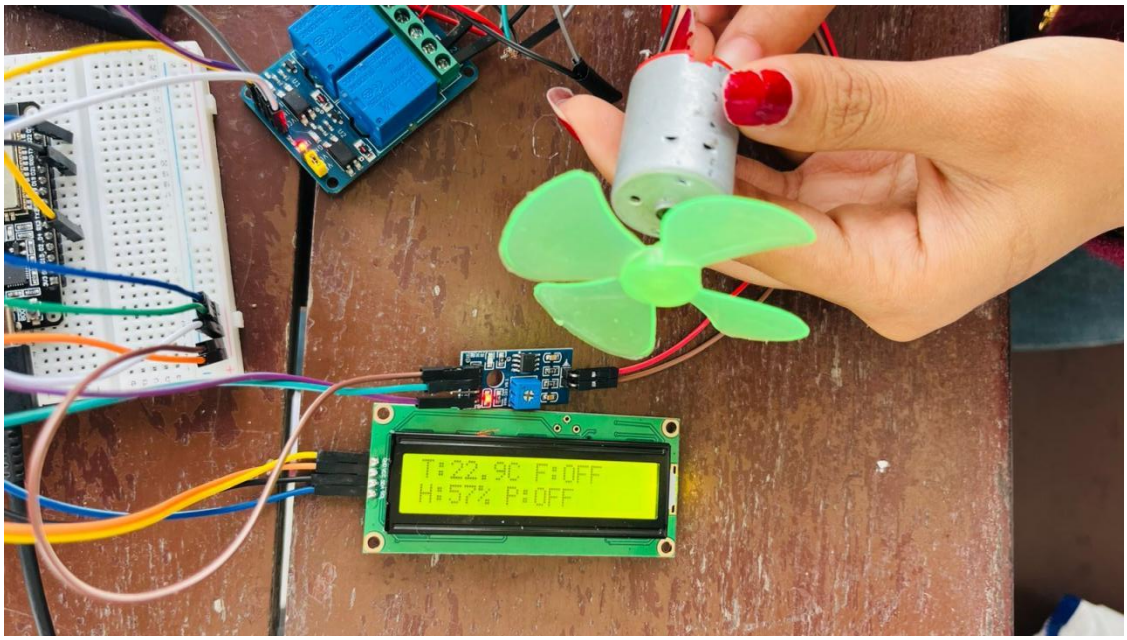


Figure 61: Successful fan off

4.19. Test no.19 - MQTT Testing

Figure 62: MQTT Testing

Objectives	Verify MQTT communication in the greenhouse.
Action	Sensor data was published in MQTT,humidity and temperature messages were received.
Expected results	Data is shown in MQTT.
Actual results	Data was shown in MQTT.

conclusion	The test was successful.

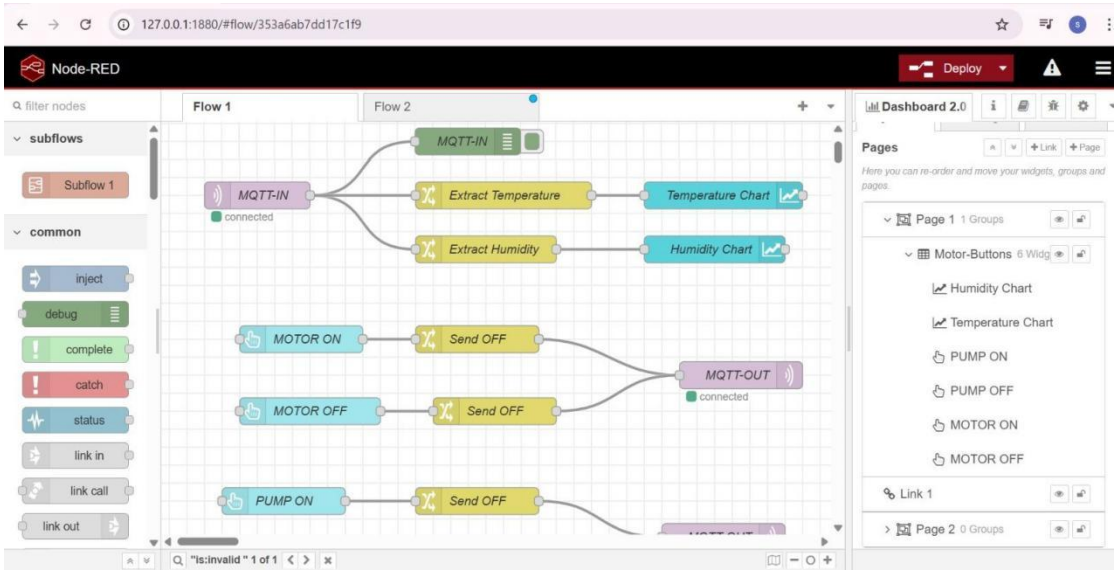


Figure 63: Node Red MQTT

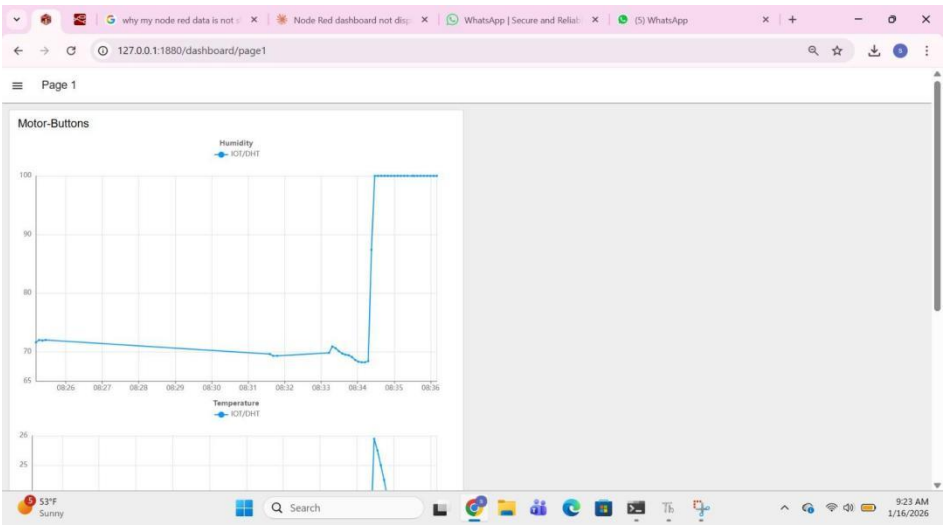


Figure 64: Humidity Graph

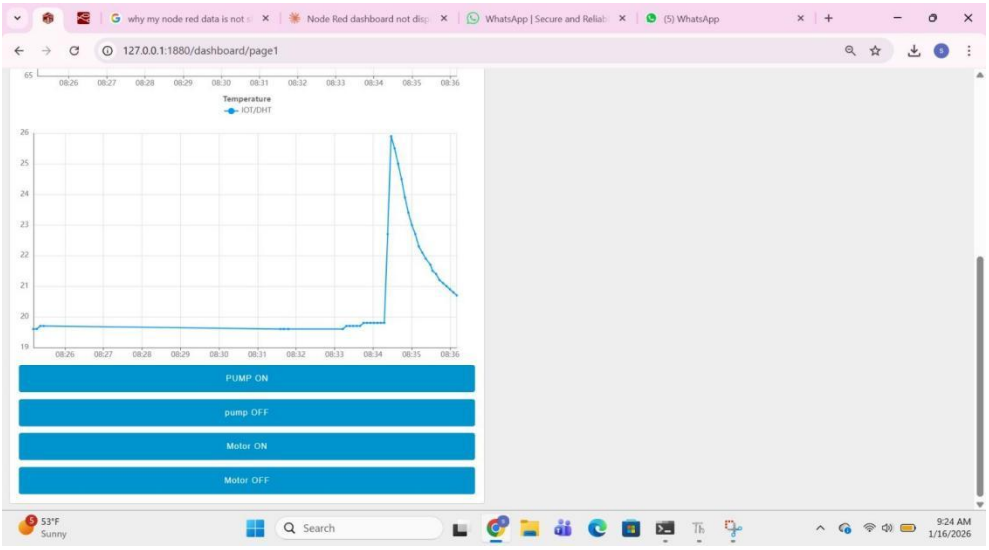


Figure 65: temperature graph

4.20. Test no.20 - automatic greenhouse

Table 19: complet automatic greenhouse overview

Objectives	Test automatic condition of greenhouse.
Action	Sensors, motor fan, water pump, relay module,battery,LCD and esp32 are assembled together and automated according to required conditions.
Expected results	Motor fan, Water pump is operated automatically from greenhouse condition and LCD displays the data.
Actual results	The motor fan, water pump were operated automatically from greenhouse condition and the LCD displayed the data.
conclusion	The test was successful.

Figure 66: soil sensor in greenhouse

Figure 67: fan turn on when temperature reach 24°c

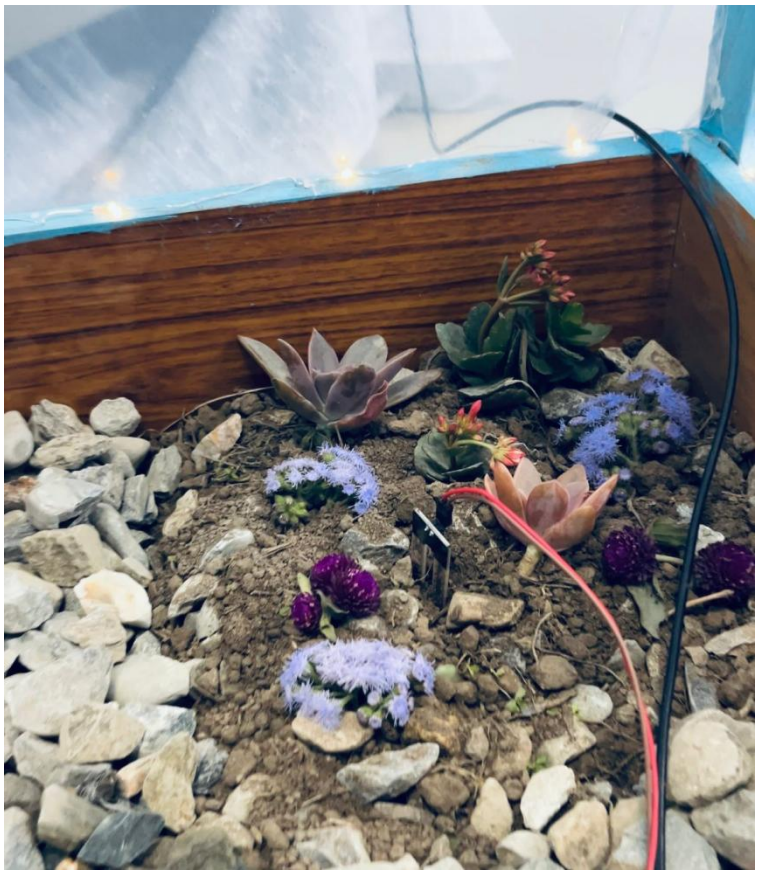


Figure 68: oil sensor in greenhouse water motor turn on when soil is dry

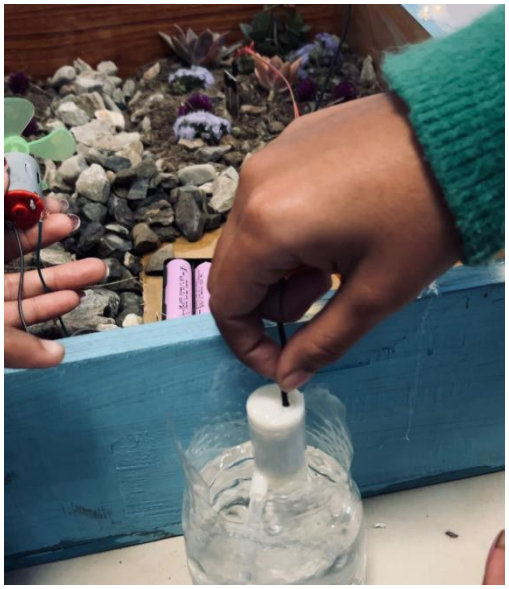


Figure 69: water motor turn on when soil is dry

Figure 70: smart greenhouse

4. Future work

As well, in future projects, the system can be improved with the addition of the IOT Integration allowing close-time monitoring and control with the help of a web or mobile platform. Sensor data, e.g. the temperature, humidity, and soil moisture can be Relocated to a cloud platform where it can be accessed remotely, analyzed and stored over the long term.

Other sensors could be add in the planning phase like light intensity, CO2 sensor and pH sensor to improve monitoring of the environment and to facilitate healthy growth of the plant. The control system may also be Enhanced to the automatic fan speed control, lighting control, and fertilizer Providing, turning the greenhouse into the completely automated one.

Design The breadboard Sample could be replaced with a PCB design as it is a more Accurate design and the wiring needs will be minimized to make the design less complicated and safe. To enhance power management, it can be Supplied with a solar power support and battery backup to keep the power on at all times in case of a power failure. Better design of enclosures can also be designed to Survive moisture, dust and heat to the components.

The additions to the software can be in the form of complex control algorithms, notification of the exceptional conditions, and the visualization of the data in the form of dashboards. Safe remote access can be achieved by provision of security features like user authentication. In general, planning and design in the future will be Intended at enhancing the scalability, efficiency, reliability, and usability of the system.Therefore, the greenhouse system will be applicable in a real-life farming setting.

5. Conclusion

This helps us to understand the conditions of plants inside the greenhouse. Micro-controllers also help us to control the devices like fans, pumps, and lights based on the readings.

In addition, this IOT project also helps us to learn about the advantages of sending real time data to a dashboard. This helps the user to check the temperature and conditions of the greenhouse even when the user is far. Unusual changes such as if the value is too high or too low alerts and notifications are sent to the user. This project also proved that IOT is very helpful even at agricultural practices by reducing the manual work and helping to create a better condition for growing plants.

By using of control elements including fans, water pumps and lights, Automatic response to sensor readings by the system was possible. As a figure, on the event that the temperature exceeds the required level, the cooling fan was turned on to scale down the surplus heat. On the same note, watering and lighting systems would be regulated depending on the soil moisture and the light conditions. Microcontrollers or control units enabled it to retrieve sensor data and control these devices effectively to guarantee the best conditions of plants.

Moreover, this IOT- related project made us realize the need of data transmission in real-time to a monitoring platform or dashboard. The data transmitted are the live sensor measurements, which can visually be viewed by a user because it is sent to a dashboard and can be viewed remotely. This remote monitoring aspect comes in handy a lot to the farmers or the owners of greenhouses that cannot always be physically present.

The other significant benefit of this system is the alert and notification. When abnormal conditions arise like too high or too low temperature, the user can be alerted

immediately by the system. It can ensure that there is rapid response that will help avoid damage to plants and enhance the safety of the crop. These timely alerts enhance quality decision making and minimize risks involved in changing the environment.

In general, this project demonstrated that IOT technology may become very useful in the sphere of agriculture. It aids in the minimization of manual labor, timesaving and efficiency in that routine jobs are automated. The intelligent greenhouse system provides an artificial environment conducive to improved growth and increase in production of plants. The project helped us to acquire practical experience in the areas of sensors, automation, real-time monitoring, and IOT applications, which are skills relevant in contemporary agriculture and smart farming systems.